



Investigating cross-market android apps: Security, protection, and components

Shishuai Yang¹ · Ruoyan Lin² · Jialong Guo² · Guangdong Bai³ · Yujia Luo² · Wenrui Diao² 

Received: 4 March 2025 / Accepted: 19 October 2025 / Published online: 19 January 2026

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

Multi-channel distribution of Android apps offers convenience to users, yet simultaneously introduces security concerns. Even though apps published in the Google Play and third-party markets share the same version code, differences in the content of apps may still arise. Notably, a recent incident involving the third-party market version of the Pinduoduo app containing malicious code highlights the intentionally-differentiated implementations of app functionalities by developers between Google Play and third-party markets. The case of Pinduoduo may be just the tip of the iceberg, underscoring the need for a comprehensive investigation into the disparities between the Google Play and third-party market versions of apps. In this work, we systematically analyzed this crucial issue, focusing on cross-market apps that claim to share the same version code. Specifically, we proposed four research questions, covering differences in app protection, security threats, permission usage, and major components. To answer these questions, we constructed a dataset containing 17,218 app pairs (filtered from 236,731 apps) and permission mappings (27,046 SDK mappings, 1,656 ContentProvider mappings, and 309 Intent mappings) for API levels 16 - 33. This dataset enables us to conduct a comprehensive differential analysis. Consequently, our investigation unveiled a series of captivating and insightful findings. Approximately 29.02% of apps show differences in one or all of the four aspects. For example, the third-party market versions of apps often request more permissions compared to their Google Play counterparts, particularly among apps from the game category. Our work can provide developers and app store operators with directions for improvement to ensure the consistency of cross-market apps, enhancing the quality of the Android apps ecosystem and user experience.

Keywords Cross-market Android apps · App protection · Security threats · Permission usage · Major components

Communicated by: Lei Ma, Roberto Pietrantuono.

Extended author information available on the last page of the article

1 Introduction

The openness of Android's app distribution channels is a key feature that distinguishes it from other mobile OSes like iOS. Android developers can release their apps not only through the official app store (Google Play), but also through various third-party markets. While this multi-channel distribution offers convenience to users across different regions, it also introduces security concerns. Apps distributed through Google Play undergo a stringent review process by the Google Play team before being released to users. In contrast, the review requirements in third-party markets are often not as rigorous as those of Google Play. Developers may thus opt to release their apps on third-party markets to circumvent Google Play's review and restrictions. This practice poses an increasing risk to users, as it opens the door to the potential download of malicious apps from third-party markets.

The case of the Pinduoduo app (Pinduoduo 2023), a popular mobile e-commerce shopping platform with approximately 900 million users, has demonstrated a real-world example of the risks associated with third-party market distribution. A recent report (Lookout 2023) revealed that its third-party market version contains malicious code that can exploit the Android OS. This code can potentially prevent users from removing the app from their devices, and can even install malicious apps in the background, remove other legitimate apps, and spy on users. In contrast, there is no evidence suggesting that the Google Play version of this app contains malicious code. The case of Pinduoduo may only be the tip of the iceberg. It highlights that the differences in implementation between the Google Play version and third-party market version of apps could be considerably significant (we will refer to Google Play as “GP”, third-party market as “3rdPM”). In our study, we focus on analyzing cross-market versions of apps that have the same version code, as such versions are expected to have consistent behaviors across all distribution channels.

To the best of our knowledge, there is currently no systematic research on the differences between the GP and 3rdPM versions of Android apps. This may be due to two difficulties in constructing the dataset: (1) Developers may not publish apps simultaneously on both GP and 3rdPM or maintain the same version. (2) Google Play currently only supports downloading the latest version of apps, not a specific version. The most relevant work was carried out by Wang et al. (2018). They mainly focus on catalog similarity across app markets, their features, publishing dynamics, and the prevalence of various forms of misbehavior (including the presence of fake, cloned, and malicious apps). In contrast, we conducted a complementary study by exploring the discrepancies between the GP and 3rdPM versions of apps, such as permissions and app protection, which have yet to be systematically studied.

Our Work In this study, we systematically evaluate the differences between the GP and 3rdPM versions of apps from four aspects: app protection, security threats, permission usage, and major components. More precisely, our objective is to answer the following four research questions:

- ⇒ **RQ1 (app protection):** *Is there any difference in the deployment of app protection measures between the GP and 3rdPM versions of apps?*
- ⇒ **RQ2 (security threats):** *Is there any difference in vulnerabilities and malware between the GP and 3rdPM versions of apps?*
- ⇒ **RQ3 (permission usage):** *Is there any difference in permission usage between the GP and 3rdPM versions of apps?*
- ⇒ **RQ4 (major components):** *Is there any difference in component usage between the GP and 3rdPM versions of apps?*

The selection of these aspects is guided by established mobile security standards, including OWASP MASVS (Mobile Application Security Verification Standard) (OWASP 2025) and SSDLC (Secure Software Development Life Cycle) (Liran-Tal 2025). App protection mechanisms (e.g., *packer*, *anti-tampering*) are essential for ensuring app integrity and intellectual property protection, aligning with MASVS-RESILIENCE and the SSDLC's asset protection goals. The analysis of security threats such as vulnerabilities and malware is highly aligned with the concerns addressed by MASVS (e.g., *MASVS-CODE*), and also conforms to the SSDLC's goal of producing secure software. Differences in permission usage are crucial for assessing privacy and platform risks, as emphasized by MASVS-PLATFORM. Lastly, variations in component usage (e.g., *Activity*, *Service*) may indicate added functionality, third-party SDK integrations, or repackaging, all of which carry security and privacy implications and are particularly relevant to the implementation and verification phases of the SSDLC.

To answer these proposed research questions, we first constructed a dataset of 236,731 3rdPM apps, with 44,283 apps also available on GP. From them, we filtered the apps whose GP version and 3rdPM version have the same version code, and 17,218 app pairs (i.e., GP and 3rdPM versions) were kept for analysis. In our analysis, we first adopted a series of heuristic methods to identify the differences in app protections, such as packer and signature verification. We then employed state-of-the-art vulnerability detection tools (such as MobSF (Ajin-Abraham 2023) and AndroBugs (Yu-Cheng 2023)) and uploaded apps to VirusTotal (VirusTotal 2023) to identify differences in security threats among the apps. For the differences in permissions, we constructed permission mappings for API levels 16 - 33 to identify SDK APIs, Content Uri's, and Intent Actions protected by permissions in apps, including 27,046 SDK mappings, 1,656 ContentProvider mappings, and 309 Intent mappings. Finally, we analyzed the differences among the four major components of apps.

Key Findings Our measurement results for the four research questions above reveal notable differences between the GP and 3rdPM versions of apps. Despite the general consistency exhibited by the GP and 3rdPM versions of most apps, those displaying differences tend to possess the following features.

- **Protection Measures Degradation.** Due to Google Play's strict security review of apps, developers tend to reduce the deployment of specific protection measures in the GP version of apps, such as packers and signature verification. On the contrary, developers often add more protection measures into the 3rdPM version of apps to enhance app security.

- **App Repackage.** The differences in the certificate's Organization fields indicate that the 3rdPM version or GP version of apps may be generated through repackaging, suggesting the possibility of copyright violation.
- **Enhanced Permission Requests.** Among different categories of apps, the 3rdPM version of game apps tends to request more system permissions, averaging 10 extra system permissions for each app. Our further investigation found that the excessive system permission requests are rarely caused by the developers. Instead, a portion is used by third-party libraries (TPLs for short), while another portion remains unused in practice. Furthermore, the 3rdPM version of apps often integrates more push services, thus introducing extra custom permissions.
- **Heightened Vulnerability and Malware Risks.** Due to excessive permission requests and the usage of TPLs, the 3rdPM version of apps is more susceptible to introducing vulnerabilities and being flagged as malware.
- **Distinctive Component Definition.** Components exclusively defined in the 3rdPM version of apps predominantly include gamepad and stub components, while those exclusive to the GP version predominantly comprise ads and Google Play Services components.

Contributions Here, we list the main contributions of this paper:

- **Systematic Differential Study.** We conduct the first systematic study on the differences between the GP and 3rdPM versions of apps from app protection, security threats, permission usage, and major components. In particular, we proposed and answered four significant research questions with sufficient supporting evidence.
- **Practical Results.** This work can enhance the overall security and transparency of the mobile app ecosystem. Furthermore, it provides valuable insights for app developers and store operators regarding app development and the regulation and governance of the mobile app market. This work can also enhance users' awareness of security.
- **Available Permission Mappings.** The raw measurement data for each research question, as well as the permission mappings we generated for Android API levels 16 - 33, are available at <https://doi.org/10.5281/zenodo.13232037>. The permission mappings will contribute to the research community, facilitating studies on apps' privacy compliance checks and malicious behavior analysis.

Remarks This work is an extension of our conference version published in the Proceedings of ISSRE 2024 (Yang et al. 2024). In the conference version, we evaluated the differences between the GP and 3rdPM versions of apps in terms of protection measures, security threats, and permission usage. In this extended version, we evaluate the differences between the GP and 3rdPM versions of apps in terms of the four defined components, as well as the differences in the declared attributes of the same components. Furthermore, we broaden our discussion to include a comprehensive discussion of the reliability of static analysis tools for large-scale empirical studies, considering various aspects such as previous research, practical applications, and industry recommendations. Additionally, we have significantly expanded the content on best practices, threats to validity, and related work.

Roadmap The remainder of this paper is organized as follows. Section 2 provides the necessary background on Android permissions, app protection measures, and the reliability of our analysis tools. Section 3 details our methodology for app collection, permission mapping generation, and differential analysis. Our findings for the four research questions (RQ1-RQ4) are presented in Section 4. We discuss the implications of our work, threats to validity, and future directions in Section 5. Section 6 reviews related work, and finally, Section 7 concludes the paper.

2 Background

In this section, we provide the necessary background of the Android permission mechanism and app protection measures, as well as discuss the reliability of static analysis tools used in this study.

2.1 Android Permission Mechanism

The permission mechanism is an Android OS-level security mechanism designed to manage the access permissions of apps to system resources and sensitive data (Google 2023g). It is based on the Linux access control model and ensures that apps can only access specific functions and data with authorization by allocating and managing permissions. In Android, permissions are categorized into different groups, each representing a specific set of system functionalities or sensitive data. For instance, permissions can control whether an app can access the camera, location information, contacts, etc. In contrast to system permissions, custom permissions are used to protect the data generated by apps and are applied to the four major components of apps.

The permissions for Android are divided into three levels: normal, dangerous, and signature. Normal-level permissions typically involve the basic functionalities of apps, and users do not need to provide explicit authorization. Once apps request these permissions, they are automatically granted. Dangerous-level permissions involve sensitive data and system functionalities, requiring user authorization at runtime. Signature-level permissions require the requesting app to be signed with the same digital certificate as the app declaring the permission, ensuring that the permissions are used by legitimate apps.

2.2 App Protection Measures

Packer App packer refers to hiding the original DEX file of the app so that the actual DEX file within the app is the packer code (Zhang et al. 2015). During app runtime, packer code often customizes the Application class to achieve the decryption and restoration of the original DEX files, as this class serves as the entry point of the app. Typically, developers will use security vendors' packer products rather than attempting to implement their own, as the packing process entails intricate technicalities and specialized expertise. Security vendors' products can deliver a heightened level of professional assurance. Therefore, detecting whether an app is packed mainly involves detecting which packer products the app has used.

APK Signature Signatures are primarily used to verify the authenticity and integrity of content. APK typically includes two types of signatures (Google 2023h): (1) Signature of the developer certificate: The developer certificate primarily includes core information such as the certificate holder, issuer, and public key, all formatted according to the X.509 standard. If two apps are published by the same developer, then the certificate's signature or holder is usually the same. (2) Signature of APK content: Developers sign the APK using a private key and embed the corresponding public key within the developer certificate. When users install or update an app, the Android OS utilizes public key to verify the APK's signature, ensuring the app has not been tampered.

Anti-debugging, Anti-VM, and Anti-root Anti-debugging, anti-VM, and anti-root are used to enhance the security of apps (Berlato and Ceccato 2020): (1) Anti-debugging. Anti-debugging is used to prevent malicious attackers from using debuggers to analyze the operation of apps, modify code, or obtain sensitive information. (2) Anti-VM. Anti-VM aims to detect whether apps are running in a virtual environment to prevent attackers from executing malicious operations or analyzing the behavior of the apps. (3) Anti-root. Anti-root aims to detect whether a device has been rooted to prevent malicious users from using root privileges to modify, tamper with, or access sensitive data in apps.

2.3 Tools

To perform differential analysis on apps, we have employed a series of state-of-the-art static analysis tools. We now describe the tools we used to demonstrate that the results generated by these tools are reliable for large-scale empirical analysis.

APKiD, MobSF, and AndroBugs APKiD (Red-Naga 2023), MobSF (Ajin-Abraham 2023), and AndroBugs (Yu-Cheng 2023) are well-known open-source security analysis tools for mobile platforms. APKiD is based on Yara rules (Victor-M.-Alvarez 2023), which can identify packer products in apps efficiently. When APKiD scans an APK, it uses pre-defined Yara rules to detect specific features in the APK, such as packer code and specific .so files. Based on these features, APKiD can accurately determine whether an app has used a certain packer. MobSF is an automated mobile app security evaluation framework capable of performing comprehensive security scans on apps. Researchers have accurately identified a range of security issues in top bank and wallet apps from India and Europe using MobSF (Ajin-Abraham 2024). Both MobSF and APKiD are mobile application security testing tools recommended by OWASP MASTG (OWASP 2024).

Similar to MobSF, AndroBugs is also a security evaluation framework. It has detected various vulnerabilities and security issues in apps from leading tech corporations, such as Microsoft, Google, and Alibaba, earning recognition in the Security Hall of Fame of these companies (Yu-Cheng 2024). Furthermore, Gao et al. (2021) scanned 454,799 apps with AndroBugs and conducted a detailed manual review of 711 of all discovered vulnerabilities, ultimately confirming that these vulnerabilities were all true positive results.

LibD and LibRadar LibD (Li et al. 2017) and LibRadar (Ma et al. 2016) use clustering-based techniques to detect TPLs. They generated unique features for each TPL by analyz-

ing numerous apps. LibD analyzed 1,427,395 apps from 45 different app markets, while LibRadar analyzed one million apps from Google Play. Based on the generated feature list, LibD and LibRadar can accurately identify TPLs in apps and resist code obfuscation. The study conducted by Zhan et al. (2020) demonstrates that LibD and LibRadar achieve high precision in detecting TPLs, with LibD at 84% and LibRadar at 98%.

3 Methodology and Dataset

This section illustrates our measurement approach and constructed datasets to answer the proposed research questions.

3.1 Methodology

As illustrated in Fig. 1, on a high level, our measurement contains three main steps, as follows:

- **App Collection.** First, we constructed the app dataset for differential analysis used in this study, including the GP and 3rdPM versions of apps.
- **Permission Mappings Generation.** Next, we generated the permission mappings for subsequent permission differential analysis, including SDK mappings, Intent mappings, and ContentProvider mappings.
- **Differential Analysis.** Based on the constructed app dataset and permission mappings, we employ a combined dynamic and static differential analysis approach to answer the proposed research questions.

3.2 App Collection

Collection of 3rdPM Version of Apps To obtain as many apps published simultaneously on GP and 3rdPM as possible, we first need to build a large app dataset from 3rdPM and then retain the apps also published on GP. We selected 10 popular 3rdPMs to crawl apps, including F-Droid, 9apps, 360, 2265, Anzhi, LapTopPCAPK, Lenovo, Leyou, Mdpda, and Uptodown. For each app market, we launched multiple processes in parallel to crawl apps simultaneously and used the app name as an identifier to determine whether it had already been crawled. While crawling apps, we recorded metadata about the apps, such as the market, category, and name. As app developers may upload apps to multiple app markets, we calculated the MD5 value of each app to remove duplicate apps. Furthermore, to guarantee that each app in the 3rdPM dataset possesses a unique version, we removed apps that shared

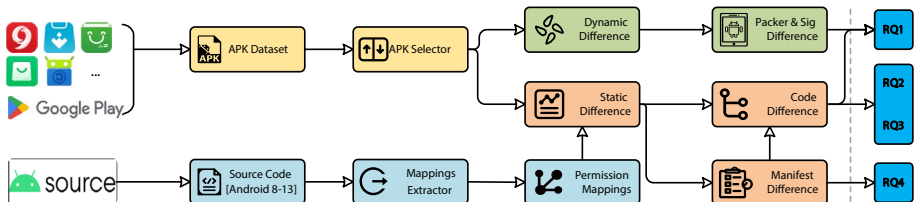


Fig. 1 Overview of the measurement flow

the same package names but had differing version codes. Some corrupted apps were also excluded. As a result, we obtained a total of 236,731 apps.

Collection of GP Version of Apps After constructing the 3rdPM dataset, we must verify whether these apps are also published on GP. The URL of the app's detail page on GP includes an id parameter, and the value of id parameter is the app's package name¹ [https://play.google.com/store/apps/details?id="package name"](https://play.google.com/store/apps/details?id='package name'). Therefore, we obtained the package names of all apps in the 3rdPM and constructed the corresponding URLs to send HTTPS requests to the GP server. Observing the status code returned from HTTPS requests, we can determine whether the app is also published on GP. If the status code is 200, the app exists on GP. Among the 236,731 apps from the 3rdPM dataset, 44,283 apps are also published on GP. After obtaining the list of apps available on 3rdPM and GP, we also recorded the version code of each app.

Since GP no longer provides an API for bulk downloading apps, we cannot directly download apps from the official GP website. Therefore, we used *Raccoon* (Raccoon 2023) to download apps from GP. Since GP cannot be accessed anonymously, we need to use a Google Account to log in to *Raccoon* before downloading apps. *Raccoon* will mimic a high-end smartphone through the device profile, connect to the GP server, and download apps. All communications between *Raccoon* and GP adhere to Google's privacy statement (Raccoon 2024), ensuring compliance with GP's guidelines and avoiding any breach of their policies. *Raccoon* also supports downloading historical versions of apps by specifying the package name and version code. However, *Raccoon* can only download one app at a time, so we must use a script to automate the apps' downloading process. Although 44,283 apps are available on both 3rdPM and GP, developers may release different versions on 3rdPM and GP. For example, developers may release version 001 of an app on 3rdPM and version 002 on GP, meaning not all 44,283 apps will have the same versions available on GP. Based on the recorded package names and version code of the 3rdPM version of apps, we use *Raccoon* to send download requests to GP for all 44,283 apps one by one. After completing the download of a certain number of apps, *Raccoon* may become unresponsive, which leads to us having to restart it periodically. Therefore, the downloading process is not continuous but has been intermittently ongoing for approximately ten days. We can only download 17,218 apps in ten days as the 3rdPM versions of many apps do not have matching versions available on GP.

Apps Filter We determine if two APK files are exactly the same by calculating their MD5 values. Apps with the same hash values are assured of having the same content. Since this work primarily focuses on apps with the same version code but exhibit differences in content, we must remove the GP and 3rdPM versions of apps with the same MD5 value from the dataset. First, we calculated the MD5 value of the GP and 3rdPM versions of APK files. Among 17,218 app pairs, the MD5 value of 33.85% is the same. As APK files are essentially ZIP compressed packages, we calculated the MD5 value of each file in APK for comparison. 37.13% of app pairs have different MD5 values of APK files but the same MD5 value for all their internal files. In this case, it may be because these two apps used different packaging methods, such as packaging tools, packaging options, or signing methods. After removing

¹ Detail Page URL Format: [https://play.google.com/store/apps/details?id="package name"](https://play.google.com/store/apps/details?id='package name')

the app pairs with the same MD5 value, our dataset is left with 4,997 app pairs for subsequent differential analysis.

3.3 Permission Mappings Generation

To analyze permission differences, we first need to identify who introduced the differing permissions between the GP and 3rdPM versions of apps. Also, it is imperative to determine if these permissions are actually being used. Therefore, we must construct permission mappings. Constructing permission mappings is crucial for the research community to conduct specific works, such as privacy compliance checks of apps. In Android, permissions are typically mapped to three entities: API Calls, Intent Actions, and Content Uri's. Generally, obtaining permission mappings is based on *Axplorer* (Erik-Derr 2023), which provides permission mappings for API levels 16 - 25. However, *Axplorer* is no longer being maintained. As a result, we cannot obtain permission mappings for API levels 26 - 33. Furthermore, permission mappings provided by *Axplorer* lack some permission mappings, such as *CAMERA* and *INTERNET*.

Permission Annotation and Comments Since Android 6.0 (API level 23), Google officially records permission mappings in two ways: (1) Using Java annotation `@RequiresPermission` ("PermissionName") to associate API with permissions. (2) Using `@link android.Manifest.permission#`"PermissionName" to describe the permissions required to call this API in the comments.

SDK Mappings Generation To determine whether the SDK APIs called within the apps are protected by permissions, we need to construct SDK mappings. Our goal is to build a more comprehensive and up-to-date set of mappings than what is currently available from *Axplorer*. Each API level corresponds to a version of the Android OS and provides a specific set of APIs for app development. Therefore, we downloaded the source code of Android 8 - 13, which includes eight Android versions: 8.0, 8.1, 9.0, 10.0, 11.0, 12.0, 12.1, and 13.0, corresponding to API levels 26 - 33, respectively. To facilitate the subsequent extraction of SDK mappings through annotations and comments (abbreviated as A&C), we extracted the Android SDK source from each of the above Android versions.

Although *Axplorer* provides SDK mappings for API levels 16 - 25, the SDK mappings are incomplete. In addition, the SDK mappings extracted through A&C for API levels 26 - 33 may also be incomplete. Therefore, we generate a more comprehensive set of SDK mappings for API levels 16 - 33 by merging the SDK mappings generated by A&C with those generated by *Axplorer*. We use the SDK mappings generation for API level 26 as an example to explain how we generate these enhanced SDK mappings for API levels 16 - 33, as shown in Fig. 2:

(1) **SDK Mappings Generation by Axplorer.** First, we generated partial SDK mappings for API level 26 based on the SDK mappings provided by *Axplorer* for API level 25. We need to check whether the APIs in the SDK mappings for API level 25 exist in the SDK source of API level 26 and whether they are deprecated. If an API is marked as deprecated, the API may be removed from the Android framework at subsequent API levels, yet

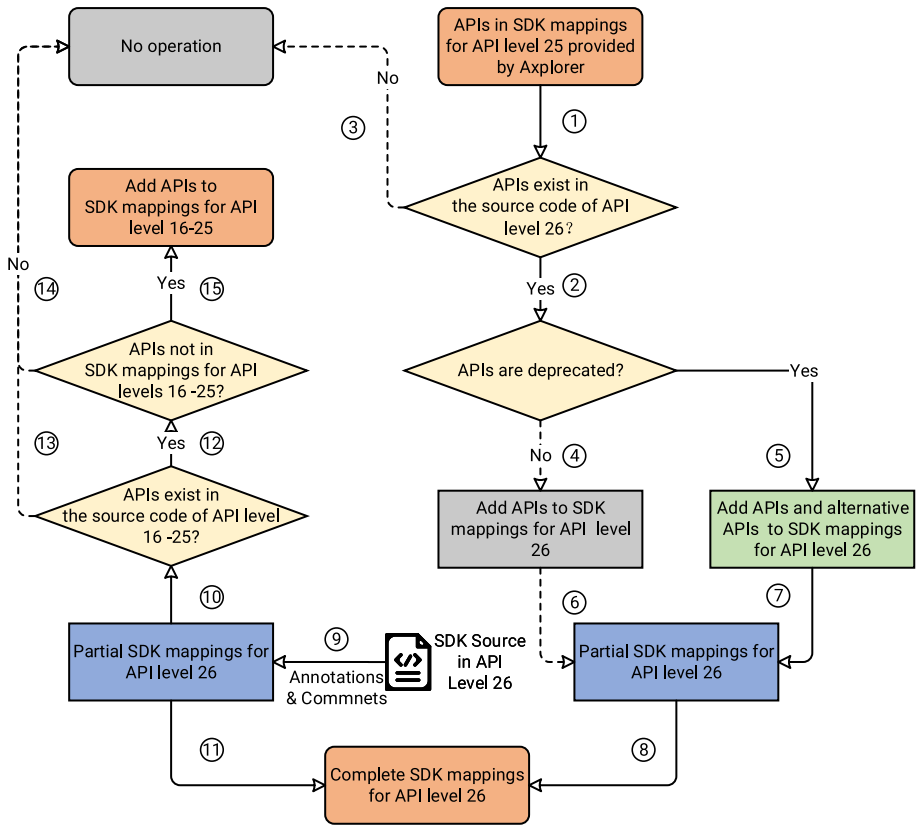


Fig. 2 SDK mappings supplement and generation

it remains accessible within the current API level. Additionally, Google typically provides alternative APIs in the comments of the deprecated API.

- If APIs in SDK mapping for API level 25 do not exist in the SDK source of API level 26, we will not perform any operation (path: ① → ③).
- If APIs in SDK mappings for API level 25 exist in the SDK source of API level 26 and are not deprecated, we will add them to the SDK mappings for API level 26 (path: ① → ② → ④ → ⑥).
- If APIs in SDK mappings for API level 25 exist in the SDK source of API level 26 but have been deprecated, we will add APIs and the alternative APIs to the SDK mappings for API level 26 (path: ① → ② → ⑤ → ⑦). For example, as shown in Listing 1, the `getDeviceId()` was deprecated in API level 26. Google recommends using the `getImei()` or `getMeid()` as alternative for `getDeviceId()` (line 1). Calling `getImei()` or `getMeid()` requires the same permission as `getDeviceId()`. Therefore, we will add the `getDeviceId()`, `getImei()`, and `getMeid()` to the SDK mappings for API level 26.

```

1  /** ... @deprecated Use (@link getImei} which returns IMEI
    for GSM or (@link getMeid} which returns MEID for
    CDMA. ... */
2  @Deprecated
3  @RequiresPermission(android.Manifest.permission.
    READ_PHONE_STATE)
4  public String getDeviceId() { .... }

```

Listing 1 Example of deprecated `getDeviceId()` in API level 26.

(2) SDK Mappings Generation and Supplement through A&C. Next, we also extract partial SDK mappings for API level 26 from the SDK source of API level 26 through A&C. If APIs in SDK mappings for API level 26 exist in the SDK source of API levels 16–25. Also, these APIs do not exist in the SDK mappings for API levels 16 - 25 provided by *Explorer*. We will add these APIs to the SDK mappings for API levels 16 - 25, thereby enriching the SDK mappings supplied by *Explorer*. (path: ⑨ → ⑩ → ⑫ → ⑮). Otherwise, we will not perform any operation. (path: ⑨ → ⑩ → ⑬ or ⑨ → ⑩ → ⑫ → ⑭)

(3) Generating Comprehensive SDK Mappings. Finally, we will generate a comprehensive set of SDK mappings for API level 26 by merging two partial SDK mappings: (a) SDK mappings generated by API level 25 (① → ② → ④ → ⑥ → ⑧ ∪ ① → ② → ⑤ → ⑦ → ⑧). (b) SDK mappings extracted from the SDK source of API level 26 through A&C (⑨ → ⑪). In addition, we will remove some duplicate SDK mappings from the two partial SDK mappings.

We supplement and generate our enhanced set of SDK mappings for API levels 16 - 33 by iterating the above steps. Notably, when using comments to extract SDK mappings, permissions may appear in the comments of APIs, but calling APIs may not require those permissions. We manually verified and excluded these APIs from the SDK mappings.

ContentProvider Mappings and Intent Mappings Generation To determine whether the `Content Uri`s and `Intent Actions` used in apps are protected by permissions, we need to construct `ContentProvider` mappings and `Intent` mappings. *Explorer* provides `ContentProvider` mappings for API levels 16 - 25. To obtain `ContentProvider` mappings for API levels 26 - 33, we retrieved all system `ContentProviders` protected by permissions from the source code of Android versions 8 - 13. For these `ContentProviders`, we extracted `Content Uri`s and their corresponding permissions. Furthermore, we also extract `Intent` mappings from the source code of `android.content.Intent` class through A&C. This class contains all `Intent Actions` along with required permissions.

3.4 Differential Analysis

After constructing the app dataset and permission mappings, we proceed to the differential analysis. To support this process, we introduce the App Selector, an automated component that systematically traverses our app collections to identify all available app pairs that share the same version code—one from the GP dataset and the other from the 3rdPM dataset. This ensures that our analysis is performed on all available app pairs. Based on the app pairs selected by the App Selector, we conduct the following analyses:

- For RQ1, we adopted a combined dynamic and static differential analysis approach to analyze the differences in app protection measures.
- For RQ2, we employed state-of-the-art vulnerability scanning tools (MobSF and AndroBugs) and uploaded apps to VirusTotal to compare the differences in vulnerabilities and malware.
- For RQ3, we thoroughly investigated the differences between the system and custom permissions used in the `AndroidManifest.xml` files of apps. Furthermore, we conducted an in-depth analysis of the sources of these differing permissions and whether the permissions are actually used.
- For RQ4, we analyzed the differences for four major components defined in `AndroidManifest.xml` files of apps, including differences in defined components and attribute values.

Experimental Setup The experiments were conducted on a server running Ubuntu 22.04, equipped with an Intel(R) Xeon(R) Gold 6226R CPU and 256 GB of RAM. The specific versions of the main analysis tools used in our study are as follows: AndroBugs v2.0.0, MobSF v3.6.0, LibRadar v1.5.0, LibD v1.0.0, and APKiD v2.1.5.

4 Findings

This section summarizes our empirical research results on the research questions proposed in Section 1.

 **RQ1.** Is there any difference in the deployment of app protection measures between the GP and 3rdPM versions of apps?

Packers We deployed APKiD to detect whether apps are packed. Furthermore, we also added some custom rules to increase the number of packers that can be detected. Among 4997 app pairs, 6.48% of 3rdPM apps are packed, compared to only 1.40% of GP apps being packed. Moreover, 5.12% of apps are exclusively packed in their 3rdPM versions, while a negligible 0.04% of apps are packed only in the GP version. The GP version of apps is more likely to reduce protection than the 3rdPM version, possibly attributable to GP's review policies. Google requires that apps must provide a unique user experience and cannot duplicate existing apps. Therefore, apps similar to existing apps on GP will be rejected from publication (Google 2023f). Google conducts rigorous reviews of various aspects of apps, such as code and resources. Packer will hide the original code of apps and only leave the packer code, which often has a high similarity across multiple packer apps. To publish apps on GP, developers may need to add additional features to differentiate them from other packed apps. This disparity in protection suggests a potential attack vector: an attacker who finds a heavily-packed 3rdPM app difficult to analyze could theoretically target its less-protected GP counterpart. Despite potential variations between the two versions, the core functionality of the app usually remains consistent.

For 1.36% of apps, both the GP and 3rdPM versions are packed. However, the level of protection provided by the packers may differ between the two versions. To assess these

differences, we define “packer strength” operationally based on the unpacking capability of FRIDA-DEXDump (Hluwa 2023). This tool is effective at recovering DEX files that are encrypted as a whole and decrypted at the initial stage of app execution. Such a packing technique, commonly referred to as whole-DEX encryption, is considered a foundational and less complex method. If an APK can be successfully unpacked using FRIDA-DEX-Dump, we classify its packer strength as relatively low. Conversely, if unpacking fails, the app is likely protected by more advanced mechanisms, such as segment-based loading or code virtualization, and is thus considered to have higher packer strength. In our dataset, 68 apps are packed in both the GP and 3rdPM versions. Therefore, we used FRIDA-DEXDump to evaluate whether the packer strength of these apps is the same. Before running FRIDA-DEXDump, we first need to launch the apps². Then, FRIDA-Dexdump can remove the packers of 48.53% of the GP and 3rdPM versions of apps simultaneously (indicating both versions utilize low strength packers), while the packers of 35.29% of the apps cannot be removed on both markets (suggesting both versions employ relatively higher strength packers that evade FRIDA-DEXDump). 16.18% of apps exhibit differences in packer strength between the GP and 3rdPM versions according to our assessment criteria. In such cases, FRIDA-DEXdump can only remove the packers on one side, meaning one version is classified as having a low strength packer while the other is classified as having a relatively higher strength packer. This discrepancy is typically attributed to the use of different packer types across markets—three of these apps are confirmed to use entirely different packers in the GP and 3rdPM versions, while the remainder likely differ in packer versions or configurations.

Table 1 presents the packer products identified in our dataset and highlights the divergent selection strategies adopted in the GP and 3rdPM ecosystems. While certain packers—such as Jiagu, Tencent, and DexProtector—are widely used across both markets, others show a clear market-specific preference. A notable example is Bangcle, which appears in several 3rdPM apps but is entirely absent from their GP counterparts. This type of discrepancy likely stems from Google’s strict transparency and compliance requirements (Google 2024). If GP’s security review determines that a specific packer product may conceal malicious behavior or employ non-compliant features, apps using it will be rejected. This pressure forces developers to make different strategic choices for each distribution channel, leading them to use different packer products—or versions—between the GP and 3rdPM releases of their apps.

Certificate Signature We extracted the SHA-1 signature of the developer certificates from the GP and 3rdPM versions of apps and compared them. Among 4,997 app pairs, 63.3% of app pairs have the same SHA-1 signature, meaning that the GP and 3rdPM versions of these apps were developed by the same developers. 36.7% of the app pairs have different SHA-1 signatures of certificates. The Subject field is used to identify the certificate holder’s information, which includes an Organization field to indicate the certificate holder’s organizational name. Therefore, we thoroughly analyzed the Organization fields for these apps with different SHA-1 signatures. Among the 1,834 app pairs with different SHA-1 signatures, 1,502 apps have the Organization field of certificates set concurrently in both the GP and 3rdPM versions. For 99.67% of apps, the Organization field of certificates differs between the GP and 3rdPM versions. This could involve two potential scenarios: (1) Developers use different certificates in different markets. For example, the

²Testing Device: The devices used for our dynamic testing are all Pixel 2 (Android 8.1.0).

Table 1 Distribution of selected packer products between the GP and 3rdPM versions of apps

Packer in GP Version	Packer in 3rdPM Version	Count
<i>Category 1: Same Packer in Both Versions</i>		
Jiagu	Jiagu	31
Tencent	Tencent	19
DexProtector	DexProtector	11
AppGuard	AppGuard	1
Ijiami	Ijiami	1
SecNeo	SecNeo	1
YiDun	YiDun	1
Subtotal		65
<i>Category 2: Packed in 3rdPM Version Only</i>		
None	Jiagu	131
None	Tencent	73
None	Bangle	38
None	Baidu	4
None	AppSealing	2
None	Ijiami	2
None	SecNeo	2
None	Others (4 packers)*	4
Subtotal		256
<i>Category 3: Packed in GP Version Only</i>		
Tencent	None	1
SecNeo	None	1
Subtotal		2
<i>Category 4: Different Packers in Both Versions</i>		
Tencent	Jiagu	2
Jiagu	Tencent	1
Subtotal		3
Total App Pairs		326

Note: "None" indicates that no packer was detected. *: "Others" includes APKProtect (1), Alibaba (1), UUSafe (1), and YiDun(1)

Organization field in the 3rdPM version of the Wireguard app³ is fdroid.org, while the GP version is Google Inc. (2) Apps on 3rdPM or GP may be generated through repackaging. For example, Organization field in the 3rdPM version of the Super-solid app⁴ is Unknown, while the GP version is Supersolid.

To further evaluate quantitatively the likelihood of repackaging in cases where the Organization field differs, we conducted an in-depth analysis using VirusTotal (VirusTotal 2023) and Avclass (AVClass 2025). Specifically, we uploaded the relevant apps to VirusTotal to retrieve multi-engine malware detection reports, and then applied Avclass to interpret the results and infer potential repackaging. An app was deemed repackaged if (1) its detection labels included characteristic keywords (e.g., *repack*); and (2) the detection names across AV engines exhibited high dispersion, defined as more than three distinct malware family names. (This heuristic is based on the observation that repackaged apps, due to obfuscation and new variant characteristics, often lead to a high

³Package Name: com.wireguard.android, Version Code: 491.

⁴Package Name: com.supersolid.honestfood, Version Code: 1000904904.

degree of naming dispersion, as different AV vendors struggle with unified identification. In contrast, conventional malware families typically have more consistent naming across engines.)

Our findings reveal a substantial disparity in the prevalence of repackaged apps across markets: 49.03% of the analyzed 3rdPM samples were flagged as repackaged, compared to just 7.82% in GP. These results not only validate the use of certificate discrepancies as a strong indicator of repackaging but also highlight repackaging as a pervasive security concern, particularly in the less-regulated third-party ecosystem, though not entirely absent from the official marketplace.

Signature Verification Signature verification means that apps check their content for tampering by verifying signatures. If the content has been tampered with, it is typically designed to interrupt the operation of the apps upon launch (pops up an error message box or crashes to the home screen). Furthermore, the code for app signature verification may exist within either the Java layer or the native layer, rendering it challenging to detect through static methods. Therefore, to analyze signature verification differences, performing dynamic testing by installing the apps on the device is necessary. We excluded some apps that would crash upon launch, either the GP or 3rdPM version, and left 2,926 app pairs for subsequent analysis. For the detection of signature verification, we employed a script that automatically repackaged the apps, applied signatures, and installed them on the device. The script will open an app and wait for 8 seconds, then use the `adb` (Android Debug Bridge) command to retrieve the current top element of the `Activity` stack. If it identifies an error prompt box or the home screen, we consider that the app has implemented signature verification.

Among 2,926 app pairs, 4.38% implement signature verification consistently across both GP and 3rdPM versions. Moreover, an exclusive adoption of signature verification in the 3rdPM versions is observed in 13.87% of the apps. In contrast, a minimal fraction, accounting for only 0.58%, enforces signature verification solely within the GP version. It may be because GP includes code similarity checks, and repackaged apps are prohibited from publishing. Some 3rdPMs lack this check, making it essential for developers to add signature verification to protect the integrity of their apps. Furthermore, an alarming 81.17% of the apps do not implement signature verification on both GP and 3rdPM versions, suggesting these apps are at significant risk of repackaging.

Anti-debugging, Anti-VM, and Anti-root We adopted `APKiD` to detect the deployment of anti-debugging and anti-VM in apps. Additionally, we used `MoBSF` to evaluate anti-root deployment. Due to the need for static code analysis for anti-debugging, anti-VM, and anti-root detection, we exclude packed apps. The deployment differences of anti-debugging, anti-VM, and anti-root between the GP and 3rdPM versions of apps are shown in Fig. 3. Most apps show consistency, with percentages of 99.25%, 99.60%, and 98.77%, respectively. These apps either simultaneously deploy protection measures in both the GP and 3rdPM versions, or they are not deployed. Particularly in the case of the anti-VM, 95.98% of apps have adopted this protection on both the GP and 3rdPM versions. Running apps on a

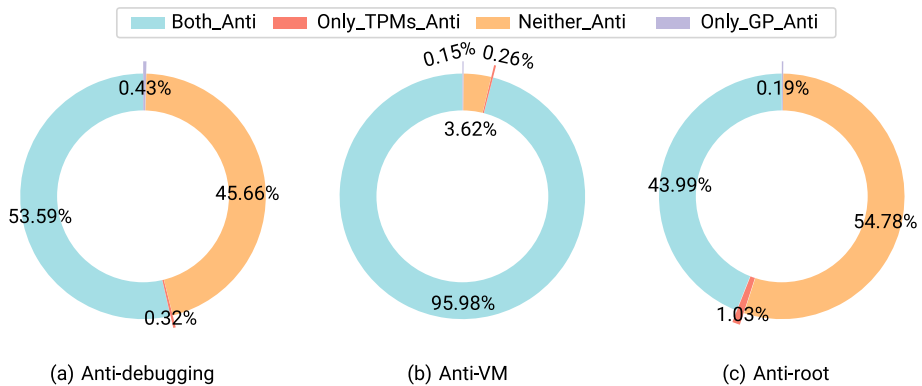


Fig. 3 Adoption differences of anti-debugging, anti-VM, and anti-root between GP and 3rdPM versions of apps

virtual machine is often the first step in performing specific app analysis, such as debugging apps, so this protection is commonly deployed in apps.

Categorical Protection Differences To analyze the variation in the deployment of protection measures across different app categories, we utilized `google-play-scraper` (JoMingyu 2023) to retrieve category metadata for each app. During the data collection process, we encountered instances where metadata was unavailable, primarily because some apps had been delisted from Google Play. This occurred for two main reasons: (1) Google Play periodically removes apps that violate its developer policies (e.g., those containing malicious code or deceptive content), and (2) developers may voluntarily discontinue maintenance due to business strategy changes or the app reaching end-of-life. As a result, we obtained category information for only 4,010 apps. The distribution of protection mechanisms across these categories is presented in Table 2.

Our findings reveal a strong correlation between an app’s protection strategy and its business nature as well as risk exposure. Tool apps stand out as an extreme case, showing the highest deployment rates across all types of protection mechanisms. This suggests that, as extensions of core system functionalities, they have become prime targets for reverse engineering and malicious code injection, prompting developers to implement the most robust defenses.

Following closely are categories such as Finance, Education, and Productivity. While these apps may not face the same level of widespread attacks as Tools, their core value lies in data, algorithms, and paid content. To safeguard these assets, developers frequently deploy anti-debugging and anti-VM technologies, effectively constructing a “data barrier” to deter information theft. Gaming apps (including Simulation, Action, Arcade, etc.) are the core reason for the discrepancy between the two markets (3rdPMs vs. GP), as their extreme reliance on packer and signature verification in 3rdPMs creates a fault-line-like contrast

Table 2 Deployment of protection measures by app category

Category	Packer		Signature Verif.		Anti-debug		Anti-vm		Anti-root	
	3rd	GP	3rdPM	GP	3rdPM	GP	3rdPM	GP	3rdPM	GP
Action	21	0	40	0	51	52	149	155	73	71
Adventure	10	0	12	0	26	24	85	86	23	22
Arcade	14	1	31	2	52	50	128	127	67	65
Art & Design	0	0	1	0	6	7	10	10	2	2
Auto & Vehicles	2	0	2	0	3	3	7	6	0	0
Beauty	1	1	0	0	9	9	11	11	2	2
Board	1	0	2	2	22	24	35	35	15	15
Books & Reference	4	3	11	10	97	98	156	156	55	55
Business	5	3	8	5	90	92	130	131	71	70
Card	0	0	0	0	11	11	18	18	5	5
Casual	16	1	27	2	55	56	144	149	56	50
Comics	0	0	0	0	12	12	15	15	11	11
Communication	1	1	3	1	17	17	28	28	13	13
Dating	0	0	0	2	18	18	22	22	11	11
Education	6	3	10	5	187	188	327	329	127	129
Educational	2	0	3	0	4	6	35	37	19	6
Entertainment	5	1	6	3	75	74	96	98	34	34
Events	0	0	0	0	2	2	4	4	1	1
Finance	7	5	8	7	90	91	117	118	74	74
Food & Drink	1	1	3	2	18	17	24	24	16	16
Health & Fitness	9	2	9	4	19	22	32	35	15	15
House & Home	3	0	4	1	17	17	22	24	15	15
Libraries & Demo	0	0	0	0	0	0	2	2	0	0
Lifestyle	8	0	10	2	46	46	73	73	38	37
Maps & Navigation	0	0	0	0	24	24	34	34	17	17
Medical	2	1	2	1	5	5	11	11	3	3
Music	1	0	6	0	8	8	26	26	10	10
Music & Audio	3	2	11	5	164	164	249	249	118	119
News & Magazines	2	2	2	2	23	23	29	29	17	18
Parenting	0	0	1	1	18	18	22	22	12	12
Personalization	0	0	2	1	131	132	197	198	84	84
Photography	1	1	1	2	28	29	40	40	19	19
Productivity	5	1	11	10	174	173	246	248	121	122
Puzzle	12	0	19	1	25	25	91	93	36	35
Racing	10	0	12	1	27	27	73	77	37	33
Role Playing	17	0	14	1	15	17	74	80	21	21
Shopping	4	0	5	0	32	32	37	38	23	23
Simulation	18	1	55	1	37	40	147	154	65	64
Social	0	0	0	0	25	25	29	29	20	20
Sports	2	0	9	0	44	44	91	92	44	43
Strategy	14	0	14	2	23	25	72	78	30	30
Tools	40	16	49	23	320	323	520	527	267	265
Travel & Local	4	2	7	5	18	18	31	32	16	16
Trivia	2	0	1	0	0	0	3	4	1	1
Video Players & Editors	3	3	3	3	25	26	35	35	14	14
Weather	0	0	0	0	41	41	53	53	31	31
Word	1	1	1	1	16	16	22	22	10	10

with the extremely low deployment on GP. For instance, the deployment rate of signature verification in Simulation games on 3rdPMs is several dozen times higher than that on GP. This directly reflects the gaming industry's core imperative to combat logic cracking (e.g., *cheating*) and pirated distribution. In contrast, app categories such as Beauty, and Food&Drink exhibit consistently low deployment of protection mechanisms across both markets, suggesting that developers perceive the security threats in these domains to be relatively low.

💡 Key Insights for RQ1

Apps distributed through 3rdPMs tend to adopt stronger protection mechanisms—such as packer and signature verification—compared to those on GP. This disparity is primarily driven by GP's stringent review policies, which compel developers to adopt differentiated protection strategies for different distribution channels. Moreover, over one-third of app pairs exhibit mismatched developer certificates, a strong indicator of repackaging behaviors that are significantly more prevalent in 3rdPMs than in GP. Protection strategy choices also exhibit strong correlations with app categories: gaming apps serve as the primary driver of cross-market differences, while tools apps are consistently protected with the highest levels of defense.

🔍 **RQ2.** Is there any difference in vulnerabilities and malware between the GP and 3rdPM versions of apps?

Differences in Vulnerabilities Apps on 3rdPM may be more prone to contain vulnerable code implementations, as the review measures of 3rdPM might not be as rigorous as GP. However, it remains unknown whether there are differences in the vulnerabilities contained in the GP and 3rdPM versions of apps. To explore these vulnerability differences, we primarily used MobSF (Ajin-Abraham 2023) for vulnerability detection. It categorizes detected vulnerabilities according to severity levels such as High, Warning, and Info. We mainly focused on the vulnerabilities classified as High and Warning levels since they pose higher security risks. To complement the coverage limitations of our primary tool, we also used AndroBugs (Yu-Cheng 2023) to expand the range of vulnerabilities identifiable in this study. In our analysis, we did not directly incorporate all results from AndroBugs, but selectively included a subset of high-risk vulnerabilities that MobSF was unable to detect. Then, we merged the distinct results from both tools and mapped the identified vulnerabilities to six categories of the OWASP Top 10 Mobile Risks (OWASP 2023), as shown in Table 3. Notably, security risks in other Top 10 Mobile Risks categories are not caused by app code. This work primarily focuses on security risks caused by app code.

High-Impact Vulnerabilities Building on this foundation, we further examined the practical security implications of the identified vulnerabilities by assessing their severity and potential exploitability. Drawing on prior security reports and related literature (OWASP 2023; MobSF 2025), we prioritized several high-impact vulnerabilities for detailed analysis. For example, Insecure WebView Implementations (`android_webview`) and Insecure SSL Implementations (`android_insecure_ssl`) are considered critical security flaws that may result in Remote Code Execution or enable Man-in-the-Middle attacks. Likewise, raw SQL queries (`android_sql_raw_query`) are susceptible to SQL injection, while hard-coded secrets (`android_hardcoded`) can lead to direct leakage of sensitive credentials. These high-severity issues exemplify the real-world risks that such vulnerabilities may pose.

Table 3 Vulnerability list for differential analysis

Owasp-Mobile	Vulnerability Name	Description	Tools
M1: Improper Platform Usage	android_webview	Insecure WebView Implementation. Execution of user-controlled code in WebView is a critical Security Hole.	MobSF
	android_webview_debug	Remote WebView debugging is enabled.	MobSF
M2: Insecure Data Storage	android_temp_file	App creates temp file. Sensitive information should never be written in a temp file.	MobSF
	android_insecure_file_mode	Use insecure file mode (MODE_WORLD_READABLE or MODE_WORLD_WRITABLE).	AndroBugs
	android_keystore_protection	The Keystores are not protected by password or use "byte array" or "hard-coded cert info" to do SSL pinning.	AndroBugs
M3: Insecure Communication	android_insecure_ssl	Insecure Implementation of SSL. Trusting all certificates or accepting self-signed certificates is a critical Security Hole.	MobSF
	android_webview_ignore_ssl	Insecure WebView Implementation. WebView ignores SSL Certificate errors and accepts any SSL Certificate.	MobSF
M5: Insufficient Cryptography	android_http	URLs that are NOT under SSL.	AndroBugs
	android_insecure_random	The App uses an insecure Random Number Generator.	MobSF
	android_sha1	SHA-1 is a weak hash with hash collisions.	MobSF
	android_md5	MD5 is a weak hash known to have hash collisions.	MobSF
	cbc_padding_oracle	This configuration is vulnerable to padding oracle attacks.	MobSF
	android_aes_ecb	The App uses ECB mode in the encryption algorithm.	MobSF
M7: Client Code Quality	android_weak_ciphers	Weak Encryption algorithm used.	MobSF
	android_sql_raw_query	App uses SQLite Database and execute raw SQL query. Untrusted user input in raw SQL queries can cause SQL Injection.	MobSF
M9: Reverse Engineering	android_hardcoded	Files may contain sensitive hardcoded information such as usernames, passwords, and keys.	MobSF

The "Vulnerability Differences" of Table 4 illustrates the differences in vulnerability contained between the GP and 3rdPM versions of apps. Most apps exhibit consistency for each vulnerability. Whether the 3rdPM or GP versions of the apps, they either both contain vulnerabilities or neither contains vulnerabilities. We have discovered some common vulnerabilities, including `android_http`, `android_insecure_random`, and `android_hardcoded`. More than 80% of the apps contained these vulnerabilities on the GP and 3rdPM versions. This is mainly due to some TPLs. For example, in the case of `android_http`, many apps use TPLs that include code implementations for cleartext traffic communication, such as `com.google.android.gms` and `org.openxmlformats.schemas`. Although GP employs a more rigorous security review process, most developers do not address the

Table 4 Differences in vulnerabilities and their responsible parties between GP and 3rdPM versions of apps

Owasp-Mobile	Vulnerability Name	Vulnerability Differences				Responsible Party (3rdPM)		Responsible Party (GP)	
		Both*	3rdPM*	GP*	Neither*	TPLs	Developer Code	TPLs	Developer Code
M1	android_webview	54.74%	0.24%	0.36%	44.66%	92.86%	7.14%	96.88%	3.12%
	android_webview_debug	29.29%	0.15%	0.24%	70.33%	81.82%	18.18%	91.67%	8.33%
M2	android_temp_file	53.86%	1.18%	0.21%	44.74%	96.00%	4.00%	92.86%	7.14%
	android_insecure_file_mode	15.69%	0.82%	0.19%	83.30%	95.31%	4.69%	100%	0%
	android_keystore_protection	22.33%	1.22%	0.17%	76.28%	100%	0%	90.0%	10.0%
M3	android_insecure_ssl	18.37%	1.24%	0.11%	80.28%	100%	0%	100%	0%
	android_webview_ignore_ssl	8.57%	0.77%	0.04%	90.62%	100%	0%	100%	0%
	android_http	90.92%	0.75%	0.13%	8.20%	96.02%	3.98%	100%	0%
M5	android_insecure_random	80.48%	1.52%	0.21%	17.79%	90.51%	9.49%	85%	15%
	android_sha1	68.44%	0.75%	0.24%	30.57%	97%	3%	100%	0%
	android_md5	63.65%	2.27%	0.26%	33.82%	95.25%	4.75%	94.12%	5.88%
	cbc_padding_oracle	33.87%	2.16%	0.06%	63.9%	98.28%	1.72%	100.0%	0%
	android_aes_ecb	11.90%	1.84%	0.13%	86.13%	99.17%	0.83%	77.78%	22.22%
	android_weak_ciphers	9.21%	1.11%	0.04%	89.64%	90.16%	9.84%	100%	0%
M7	android_sql_raw_query	73.6%	0.75%	0.15%	25.5%	98.83%	1.17%	100%	0%
M9	android_hardcoded	80.28%	1.97%	0.09%	17.66%	98.94%	1.06%	91.23%	8.77%

*: “Both” indicates that both the GP and 3rdPM versions contain vulnerabilities, “3rdPM” indicates that only the 3rdPM version has vulnerabilities, “GP” indicates that only the GP version has vulnerabilities, and “Neither” represents that neither version has vulnerabilities

security vulnerabilities of the 3rdPM versions when publishing the GP versions of apps. Meanwhile, the occurrence rates of some vulnerabilities are relatively low, such as `android_insecure_file_mode`, `android_insecure_ssl`, `android_webview_ignore_ssl`, `android_aes_ecb`, and `android_weak_ciphers`. More than 80% of the apps on both the GP and 3rdPM versions do not contain these vulnerabilities.

For each vulnerability, a small portion of apps exhibit differences, meaning that the vulnerabilities are present only in the GP or 3rdPM version. The proportion of differences in vulnerability contained between the GP and 3rdPM versions of apps ranges from 0.39% - 2.53%. The `android_webview_debug` vulnerability exhibits the smallest difference, while the `android_md5` vulnerability exhibits the largest difference. As shown in Table 4, most high-impact vulnerabilities, such as `android_insecure_ssl`, `android_sql_raw_query`, and `android_hardcoded`, are more frequently found exclusively in 3rdPM versions than in their GP counterparts. For instance, the `android_hardcoded` vulnerability appears only in the 3rdPM version in 1.97% of app pairs, compared to just 0.09% where it appears only in the GP version. Although the absolute percentages are relatively low, the consistent pattern across multiple vulnerabilities suggests a systemic issue in 3rdPM ecosystems. A notable exception occurs in the M1 category, particularly for the `android_webview` vulnerability, where the GP version is slightly more likely to exclusively contain the issue (0.36%) compared to the 3rdPM version (0.24%). Despite such outliers, the broader trend clearly indicates that critical security issues are more prevalent in 3rdPM versions, which may reflect weaker review standards and create a larger potential attack surface in these alternative distribution channels.

To understand the causes behind the differences in vulnerabilities, it’s crucial to determine the source of the vulnerabilities—whether they stem from developers’ coding practices or the integration of TPLs. We mainly used `Libd` (Li et al. 2017) to identify TPLs. Additionally, we used the TPL feature files provided by `LibRadar` (Ma et al. 2016) and manually added some TPLs to the feature file to supplement the number of TPLs we can identify. This is outlined in the “Responsible Party” of Table 4. The differences in vulnerabilities contained in the 3rdPM and GP versions of apps are primarily caused by TPLs,

and developers rarely actively introduce vulnerabilities. In the case of `android_aes_ecb`, the two versions of apps exhibited the largest difference. In the 3rdPM version of apps, only 0.83% of vulnerabilities were introduced by developer code, whereas in the GP version, 22.22% of vulnerabilities were introduced by developer code. After conducting reverse engineering on the apps, the primary reason is that the apps added BLE (Bluetooth Low Energy) related functionalities in the GP version, such as device discovery and connection management. However, the AES algorithm with ECB mode was used for communication encryption.

Differences in Malware Although most of the GP and 3rdPM versions of apps exhibit similarity in each of the vulnerabilities mentioned above, it does not mean that the differences in malware detection are also consistent. VirusTotal (VirusTotal 2023) is an online malware detection service incorporating dozens of antivirus engines, so we uploaded the apps to VirusTotal to obtain the differences in malware. If any antivirus engine identifies the apps as malicious, then we consider apps to be malicious. To reduce the likelihood of false positives—especially those caused by the presence of packed apps, which are often misclassified as malicious—we excluded all apps previously identified as packed based on the results of our previous packer differential analysis. Therefore, the following detection statistics are based exclusively on non-packed apps. Based on this filtering, 17.19% of the 3rdPM version of apps are identified as malicious, while the GP version is benign. Only 0.94% of the GP versions of apps are identified as malicious, while the 3rdPM versions are benign. 6.27% (293) of the GP and 3rdPM versions of apps are both identified as malicious. Among these 293 apps, the 3rdPM version is typically identified as malicious by an average of six antivirus engines, whereas the GP version is identified as malicious by an average of two antivirus engines. Malware detection discrepancies between the GP and 3rdPM versions of apps are notably greater than vulnerability detection. 3rdPM versions are more frequently identified as malicious or flagged by more antivirus engines.

This indicates that integrating specific TPLs or requesting excessive permissions (described in RQ3) in the 3rdPM version of apps is more likely to be recognized as malware by antivirus engines. Taking the app `RedHands`⁵ as an example, its 3rdPM version exhibits significant security differences compared to the GP version: the former is flagged as malicious by 39 antivirus engines, whereas the latter is not reported as a threat by any engine. A deeper analysis reveals that the 3rdPM version integrates `DroidPlugin` (DroidPlugin 2025), a plugin-based framework. This framework allows the app to dynamically load and execute external code modules that have not undergone security vetting—a common technique used by malware to evade static analysis. In addition, this version requests several high-risk permissions that empower dynamically loaded malicious code to perform harmful actions (Dai et al. 2020). For example, `SYSTEM_ALERT_WINDOW` (which can be used for phishing), `READ_EXTERNAL_STORAGE` (which can steal private data), and `WRITE_SETTINGS` (which can alter system settings). It is precisely this combination of dynamic code loading and dangerous permissions that forms a clear malicious behavior pattern, ultimately leading to the app being flagged as malware by numerous engines on VirusTotal.

⁵Package Name: `com.redhands.twoplayergames`, Version Code: 4.

💡 Key Insights for RQ2

While most high-risk vulnerabilities are present equally in both GP and 3rdPM versions—indicating developers often don't patch these issues for GP release and that TPLs are the primary source—the difference in malware detection is stark. 3rdPM apps are flagged as malicious far more frequently (17.19% vs. 0.94%) and by more AV engines, even after excluding packer apps to avoid false positives. This confirms that while vulnerability landscapes are similar, the specific combination of TPLs and excessive permissions in 3rdPM apps triggers malware heuristics much more often, establishing these markets as significant vectors for malware distribution.

🔍 **RQ3.** Is there any difference in permission usage between the GP and 3rdPM versions of apps?

Differences in System Permissions Firstly, we extracted the `AndroidManifest.xml` files of the GP and 3rdPM versions of apps for differential analysis. Among 4,671 app pairs, 13.17% of apps in the 3rdPM version request system permissions that are not present in the GP version, averaging 10 extra system permissions for each app. 0.64% of apps in the GP version request system permissions that are not present in the 3rdPM version, averaging 2 extra system permissions for each app. 0.56% of apps requested unique permissions in both the GP and 3rdPM versions. Many apps request more system permissions in the 3rdPM version than in the GP version, which may be attributed to Google's permission declaration form mechanism (Google 2023b). When developers want to publish apps on GP, it is necessary to submit the permission declaration form to the GP team. If permissions are requested within the apps but are not listed in the permission declaration form, then apps will not be allowed to be published. The GP team conducts reviews of apps' core functionalities to ensure that the permissions requested by the apps are necessary for their intended usage scenarios and should not include unnecessary permissions. On the other hand, 3rdPM may enforce more relaxed scrutiny on permissions.

Sources Analysis Since many 3rdPM versions of apps request more system permissions than the GP version, we primarily focus on two questions: (1) Do the extra requested permissions in the 3rdPM version actually get used? (2) If the extra requested permissions are used, who uses them – developer code or TPLs?

After extracting permission mappings (Section 3.3), we performed a static analysis on the 3rdPM version of apps that request extra permissions using AndroGuard (Androguard 2023). We retrieved the SDK APIs, Intent Actions, and Content Uris used in these apps and recorded their usage locations. Notably, apps typically use Content Uris in two ways: (1) Directly using the Content Uris string. (2) Utilizing pre-defined Content Uris constant provided by the Android OS. For the recognition of Content Uris constants, we need to pre-extract the mappings of Content Uris constant to Content Uris string from the SDK source. Then, using these mappings, we can convert Content Uris constant into the corresponding Content Uris string. Subsequently, to determine

whether the SDK APIs, Intent Actions, and Content Uris used in the apps are protected by permissions, we selected the corresponding API level permission mappings based on each app's `targetSdkVersion` for comparison.

To determine the caller (developer code or TPLs) of SDK APIs, Intent Actions, and Content Uris, we must identify the TPLs used in the apps through `Libd` and `LibRadar`. Specifically, this attribution process is primarily based on package name analysis, a common heuristic used by TPL detection tools. Both `Libd` and `LibRadar` maintain extensive databases of signatures for known TPLs. These signatures are largely composed of the package names associated with the libraries (e.g., *com.google.android.gms* for *Google Play Services*). During our static analysis, when we identify a method call that requires a permission, we inspect the package name of the class containing that method. If the package name matches a known TPL signature from the databases, we attribute the permission usage to that TPL. Conversely, any code located in a package name that does not match any TPL signature is considered first-party code and thus attributed to the developer.

For each 3rdPM version of the app, we extracted lists of extra requested permissions, permissions used by developer code, and permissions used by TPLs. By comparing these three lists, if an app's extra requested permissions are not present in either the permissions list requested by developer code or the permissions list requested by TPLs, we consider permissions to be over-claim, indicating that permissions are requested but not used. The percentage of extra requested permissions in 3rdPM versions of apps introduced by developer code, TPLs, and over-claim, as shown in Fig. 4. This figure primarily explains the sources of the extra requested permissions in the 3rdPM versions, which can be attributed to developer code, TPLs, or over-claim. For example, "579" indicates that 579 apps added extra permissions in the 3rdPM version, with only 0 - 10% of these permissions introduced by developer code in each app. Therefore, among 579 apps, most of these extra requested permissions are introduced by TPLs or over-claim.

The CDF (cumulative distribution function) of the usage count of extra requested permissions in the 3rdPM version of apps among developer code, TPLs, and over-claim is illustrated in Fig. 5. The CDF also reveals that in the 3rdPM version of apps, the extra requested permissions are seldom introduced by developer code. The distribution of permissions introduced by developer code ranges from 0 to 5. In 89.86% apps, developers introduce 0 permissions. The distribution of permissions introduced by TPLs ranges from 0 to 10. In 73.01% of apps, the number of permissions introduced by TPLs is less than or

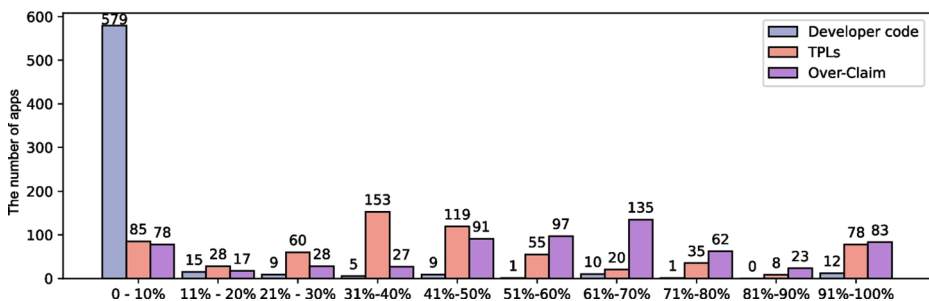


Fig. 4 Percentage distribution of extra requested permissions in 3rdPM versions of apps introduced by developer code, TPLs, and over-claim

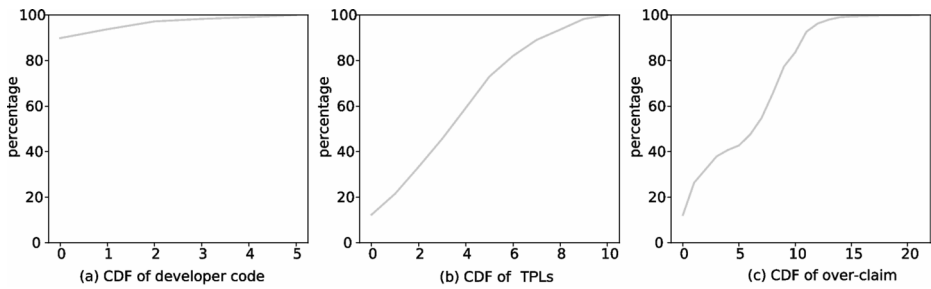


Fig. 5 CDF of the usage count of extra requested permissions in the 3rdPM version of apps among developer code, TPLs, and over - claim

equal to 5. The distribution of permissions for over-claim from 0 to 21. In 92.67% of apps, the number of over-claim permissions is less than or equal to 11.

87.83% of apps have over-claim permissions. This indicates that in most 3rdPM versions of apps, many extra requested permissions might not be used in practice. We have analyzed the distribution of `protectionLevel` for over-claim permissions. The `protectionLevel` for most over-claim permissions fall under the `normal` and `dangerous`, accounting for approximately 71.42%. The normal-level permissions are automatically granted to apps without requiring explicit requests. However, user authorization is needed for the dangerous-level permissions. Furthermore, a portion of over-claimed permissions with `protectionLevel` is categorized as `Signature` or `SignatureOrSystem`, accounting for approximately 29.58%. Although many 3rdPM apps request these permissions, they usually cannot obtain such permissions.

Granting excessive permissions in an app can create serious security vulnerabilities, opening the door for malicious exploitation by third-party components. A typical real-world case is the `SimBad` malicious ad library incident (Check-Point 2025). In this incident, over 200 apps (with more than 150 million total downloads) requested excessive permissions that were not necessary for their core functions, such as `SYSTEM_ALERT_WINDOW` and `RECEIVE_BOOT_COMPLETED`, to integrate ad functionality. A third-party ad SDK named `SimBad` then exploited these granted permissions to display intrusive ads in the background, redirect users to phishing websites, and even induce them to download other malware, all without the user's knowledge.

Attribution Analysis of Permission Inflation in Game Apps Next, we aim to confirm whether the differences in permission requests between the GP and 3rdPM versions of apps are related to the category of the apps. Therefore, we used the `google-play-scraper` (JoMingyu 2023) to retrieve metadata for these apps that exhibit differing permission requests from GP. The category distribution for these apps is illustrated in Fig. 6. Whether in 3rdPM or GP, apps categorized as game tend to request more permissions. Game apps may need to access various hardware resources on the device, such as camera, microphone, and storage. This access is necessary to create specific gaming experiences like augmented reality features, voice chat, and game recording. Retaining excessive permissions in benign apps can be exploited by malicious apps. For instance, malicious apps could execute the confused deputy attack by leveraging permissions present in benign apps (Dietz et al. 2011). This

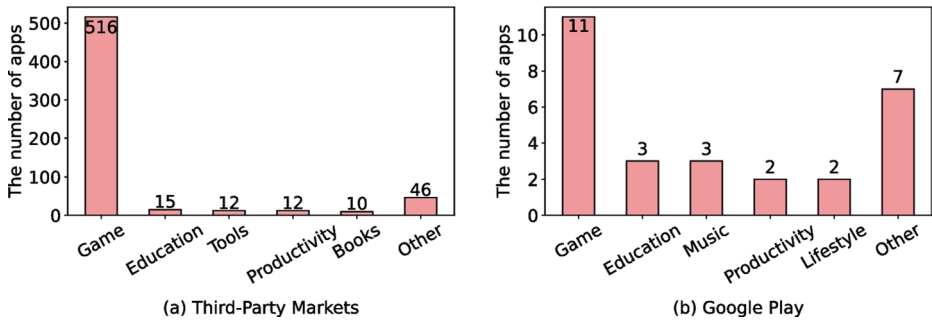


Fig. 6 Category distribution of apps that requested extra permissions in the GP or 3rdPM version.

indicates that game apps should require more attention and scrutiny regarding security and privacy, especially permission control.

Given the prevalence of extensive permission requests in game apps, we focus on analyzing third-party libraries to understand their role in this permission inflation. Our analysis confirms that this trend is strongly linked to the inclusion of specific, permission-heavy SDKs, particularly for advertising and user analytics. Developers often include these TPLs to monetize their apps, but in doing so, they also bundle in the extensive permissions requested by these libraries, even if the permissions are not essential for the core gameplay. To illustrate this phenomenon, we identified the most frequently used TPLs in game apps within our dataset. Table 5 presents the top 10 TPLs related to advertising, analytics, and game engines, detailing their usage counts and the typical permissions they request.

As the table clearly shows, a majority of the top-ranked libraries are ad or analytics SDKs. For example, the Unity Ads SDK (`com.unity3d.ads`), which appears in 236 of our game samples, requests multiple sensitive permissions, including precise location (`ACCESS_FINE_LOCATION`), phone state (`READ_PHONE_STATE`), and external storage access (`WRITE_EXTERNAL_STORAGE`). For the `READ_PHONE_STATE` permission, prior to Android 10, the Unity Ads SDK primarily used it to access hardware identifiers (e.g., IMEI) for generating ads identifiers or user tracking IDs. However, starting from Android 10 (API 29), Google has significantly restricted access to `READ_PHONE_STATE` (e.g., retrieving the IMEI now requires special permissions and is no longer available to ordinary apps) (Google 2025a). As a result, many ads SDKs continue to request this permission largely for historical or backward compatibility reasons rather than out of strict necessity. Another concrete example is the Tapjoy SDK, which even requests the `BLUETOOTH` permission. This analysis provides a direct causal link explaining why game apps often request permissions that seem unrelated to their core functionality.

Differences in Custom Permissions Among 4,671 app pairs, 1.28% of apps declare custom permissions in the 3rdPM version that are not present in the GP version, averaging 2 extra custom permissions for each app. 0.15% of apps in the GP version declare custom permissions that are not present in the 3rdPM version, averaging 1 extra custom permissions for each app. 0.09% of apps declare unique custom permissions in both the GP and 3rdPM versions. As the prefix of custom permissions typically corresponds to the package name

Table 5 Top 10 TPLs in 3rdPM version of game apps and their requested permissions

Rank	TPL	Category	Usage Count	Typical Requested Permissions
1	com.unity3d.player	Game Engine	274	ACCESS_FINE_LOCATION, READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE, ACCESS_NETWORK_STATE
2	com.unity3d.ads	Ads SDK	236	ACCESS_FINE_LOCATION, READ_PHONE_STATE, WRITE_EXTERNAL_STORAGE, ACCESS_NETWORK_STATE
3	com.chartboost.sdk	Ads SDK	140	ACCESS_WIFI_STATE, ACCESS_NETWORK_STATE
4	com.facebook	Social/Ads/Analytics	131	ACCESS_NETWORK_STATE, WAKE_LOCK
5	com.google.firebase	Analytics/Backend	121	BROADCAST_STICKY, ACCESS_NETWORK_STATE
6	com.moat.analytics	Analytics SDK	112	ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION
7	com.applovin.impl	Ads SDK	110	ACCESS_FINE_LOCATION, WRITE_EXTERNAL_STORAGE, ACCESS_NETWORK_STATE
8	com.adcolony.sdk	Ads SDK	82	ACCESS_FINE_LOCATION, VIBRATE, WRITE_EXTERNAL_STORAGE, ACCESS_NETWORK_STATE
9	com.ironsource.mediationsdk	Ads Mediation	80	ACCESS_FINE_LOCATION, WRITE_EXTERNAL_STORAGE, ACCESS_NETWORK_STATE
10	com.tapjoy	Ads/Offerwall	61	READ_PHONE_STATE, BLUETOOTH, ACCESS_FINE_LOCATION, WRITE_EXTERNAL_STORAGE

of apps, we have removed the package name prefix of custom permissions. In the 3rdPMs versions of apps, the names and counts of the top 5 extra defined custom permissions are: (PUSH_PROVIDER, 24), (PUSH_WRITE_PROVIDER, 24), (PROCESS_PUSH_MSG, 24), (TT_PANGOLIN, 19), and (C2D_MESSAGE, 7).

These extra defined custom permissions are required mainly by third-party push services. App developers must declare these custom permissions in the manifest file; otherwise, they cannot use these third-party push services. To attract more users and increase the app's exposure, the 3rdPM may encourage developers to use push services to regularly send users notifications, reminders, and promotional information, thereby increasing the app's user activity and retention rate. For example, the 3rdPM version of Italki app⁶ uses two push services, JPUSH (URORA 2023) and C2DM (Google 2023a), while the GP version does not use any push service.

⁶Package: com.italki.app, Version Code: 21256.

💡 Key Insights for RQ3

Apps from 3rdPMs exhibit notable permission inflation, requesting on average 10 more system permissions than their GP counterparts. These additional permissions are primarily introduced by TPLs and, critically, include a substantial number of *over-claimed* permissions. This issue is widespread: 87.83% of 3rdPM apps with extra permissions contain such over-claims, introducing substantial risk for privilege abuse. Moreover, differences in custom permission usage reflect divergent business models, with 3rdPM-specific permissions frequently linked to third-party push services designed to enhance user engagement.

🔍 **RQ4.** Is there any difference in component usage between the GP and 3rdPM versions of apps?

Overall Statistics The differences in the definitions of the four major components between the GP and 3rdPM versions of apps, as shown in Table 6. Compared to Receiver and Provider, the differences in the 3rdPM version of apps in Activity and Service are more significant. Among 4,997 app pairs, 17.97% of the 3rdPM version of apps define Activity that are not defined in the GP version, averaging 10 extra Activities per app. Additionally, 13.95% of the 3rdPM version of apps define Services that are not defined in the GP version, averaging 5 extra Services per app. The GP versions of apps rarely define the four major components that are not defined in the 3rdPM version, with a proportion ranging from 1.96% to 5.72%. In most cases (accounting for over 80%), the components defined in the 3rdPM version of apps were entirely the same as the GP version. The 3rdPM versions of apps tend to expand functionality or modify behavior by adding Activity and Service components, whereas Google Play versions adhere to stricter component definition standards.

Differences in Components We extract the defined Activity, Service, Receiver, and Provider from the `AndroidManifest.xml` files of each app's 3rdPM and GP versions, respectively. To identify the top 5 components, we perform differential analysis on the four major components extracted separately, accumulating the components uniquely defined in either the 3rdPM or GP version of the app. The top 5 components among the four major components are defined exclusively in either the GP or 3rdPM version of apps, as shown in

Table 6 Differences in the definitions of four major components between the GP and 3rdPM versions of apps

Name	3rdPM-Unique*	GP-Unique [†]	Both-Unique [‡]	Both-Same [◇]
Activity	13.93%	1.68%	4.04%	80.35%
Service	12.71%	1.34%	1.24%	84.71%
Receiver	3.68%	0.90%	1.06%	94.36%
Provider	4.12%	1.46%	1.0%	93.42%

*: "3rdPM-Unique": Components defined in the 3rdPM version but not in the GP version

Table 7. The four major components defined exclusively in the 3rdPM version of apps can be mainly categorized into the following four types:

- **Gamepad Components.** “kuaiyouxi” is a game service SDK designed for the Chinese market. Developers often integrate such SDKs in the China-specific versions of their apps to cater to local user preferences, offering enhanced game controller support, community features, or game acceleration services.
- **Stub Components (IBM 2023).** “Stub Activities” and “Stub Services” are typically empty components used for testing or as placeholders. Reverse engineering reveals that these components contain no actual functional code, indicating they are defined solely for placeholder purposes. To prevent app piracy or reverse engineering, developers commonly adopt packer techniques. These stub components are often associated with the packer process or dynamic loading frameworks, where the real code is loaded only at runtime.
- **Huawei Components (Huawei 2023).** The component names starting with “com.huawei” all belong to the HMS (Huawei Mobile Services) framework. They play roles in various aspects, such as push notifications, analytics, updates, and initialization within the HMS ecosystem. Due to the unavailability of Google Mobile Services (GMS) in certain regions (e.g., *China*), developers adopt HMS as an alternative.
- **Misc Components.** DownloadReceiver, TTFileProvider, and TTMultiProvider are components of the Pangle SDK (ByteDance 2025), a global mobile advertising platform developed by ByteDance. Possibly due to the unavailability of GMS, some apps in 3rdPMs use third-party ad libraries for monetization instead of Google AdMob. FileProvider (Google 2025b) is an official Android OS component that enables secure file sharing between apps, a feature that many third-party SDKs depend on. For instance, the Pangle SDK utilizes FileProvider to securely manage and share its downloaded ad assets.

Similarly, the four major components defined exclusively in the GP version of apps can also be categorized into the following four types:

Table 7 Top 5 components defined exclusively in either the GP or 3rdPM version of apps

Name	3rdPM-Unique	Total	GP-Unique	Total
Activity	com.kuaiyouxi.gamepad.sdk.shell.KyxShellActivity	249	com.google.android.gms.ads.AdActivity	132
	com.kuaiyouxi.gamepad.sdk.LauncherActivity	249	com.unity3d.services.ads.adunit.AdUnitActivity	55
	com.kuaiyouxi.gamepad.sdk.KyxSDKActivity	249	com.unity3d.services.ads.adunit.AdUnitTransparentActivity	55
	com.xiaolu123.stub.Activity00	244	com.unity3d.services.ads.adunit.AdUnitTransparentSoftwareActivity	55
	com.xiaolu123.stub.Activity01	244	com.unity3d.services.ads.adunit.AdUnitSoftwareActivity	55
Service	com.xiaolu123.stub.Service00	244	com.google.android.gms.ads.AdService	43
	com.xiaolu123.stub.Service01	244	com.google.android.gms.measurement.AppMeasurementService	41
	com.xiaolu123.stub.Service02	244	com.google.android.gms.measurement.AppMeasurementJobService	41
	com.xiaolu123.stub.Service03	244	com.google.firebase.components.ComponentDiscoveryService	35
	com.xiaolu123.stub.Service04	244	com.google.firebase.messaging.FirebaseMessagingService	32
Receiver	com.ss.android.downloadlib.core.download.DownloadReceiver	72	com.google.android.gms.measurement.AppMeasurementReceiver	41
	com.huawei.hms.support.api.push.PushMsgReceiver	26	com.google.firebase.iid.FirebaseInstanceIdReceiver	38
	com.huawei.hms.support.api.push.PushReceiver	26	com.google.android.datatransport.runtime.scheduling.jobscheduling.AlarmManagerSchedulerBroadcastReceiver	26
	com.huawei.hms.analytics.receiver.HiAnalyticsSvcEvtReceiver	18	com.google.android.gms.measurement.AppMeasurementInstallReferrerReceiver	21
	com.bangle.everisk.stub.AlarmReceiver	14	androidx.work.impl.utils.ForceStopRunnable\$BroadcastReceiver	14
Provider	android.support.v4.content.FileProvider	76	com.google.android.gms.ads.MobileAdsInitProvider	60
	com.bytedance.sdk.openadsdk.TTFileProvider	62	com.facebook.ads.AudienceNetworkContentProvider	38
	com.bytedance.sdk.openadsdk.multipro.TTMultiProvider	43	com.google.firebase.provider.FirebaseInitProvider	36
	com.huawei.hms.update.provider.UpdateProvider	33	com.bytedance.sdk.openadsdk.multipro.TTMultiProvider	11
	com.huawei.agconnect.core.provider.AGConnectInitializeProvider	32	androidx.startup.InitializationProvider	9

- Ads Components (Facebook 2023; Unity 2023; Google 2023d). These are SDK components from Google AdMob, Facebook Audience Network, and Unity Ads, which are the most mainstream ad monetization tools in the global market. They rely on GMS and are designed for global users.
- Google Measurement Components (Google 2023e). These components belong to the core services of Google Analytics for Firebase and are used to collect and analyze user behavior data. It is part of the GMS suite, and in some 3rdPM versions, it is replaced by local services such as HMS Analytics or Umeng Analytics (Umeng 2025).
- Google Firebase Components (Google 2023c). Component names containing “firebase” are associated with the Firebase platform. These components play roles in various aspects of Firebase services, such as component discovery, messaging, instance ID handling, and initialization. Due to its reliance on GMS, it may be unavailable in some regions.
- Misc Components. `AlarmManagerSchedulerBroadcastReceiver`, `ForceStopRunnable$BroadcastReceiver`, `TTMultiProvider`, and `InitializationProvider` originate from four different libraries, with functions covering Google’s data transport, official Android libraries for background tasks and startup optimization, as well as ByteDance’s third-party advertising service.

The architecture and components of an app exhibit significant differences between markets. These are often necessary strategic adjustments made to adapt to each region’s unique technical ecosystem, business environment, and legal regulations. From a technical perspective, when a market lacks a globally prevalent mobile service ecosystem, developers must integrate localized alternatives to implement core functionalities. From a business perspective, to effectively acquire users and achieve monetization, the app needs to connect with locally dominant social, payment, and advertising channels. From a regulatory perspective, strict data sovereignty and privacy laws in certain regions also require the app to make corresponding technical adjustments for compliance. For example, in the Chinese market, the widespread absence of GMS often requires developers to integrate HMS or other local SDKs instead.

Differences in Attribute Values In addition to differences in component definitions, even if the GP and 3rdPM versions of apps define components with the same name, the internal attribute values they contain may still exhibit significant variations. We conducted a detailed differential analysis on these attribute values, and the top 10 differing attribute values in four major components between the GP and 3rdPM versions of apps are shown in Table 8. Most of the attribute value differences are concentrated in the `Activity` component. In the runtime logic of the `Activity`, these differences can lead to variations in interface display effects, interaction methods, and other aspects. For example, differences in the `launchMode` attribute value may change the launch mode of the `Activity`, which in turn affects the app’s task stack management and user operation flow. In comparison, the attribute value differences in the `Service`, `Receiver`, and `Provider` components are relatively fewer.

This principle of market-driven adaptation also extends to the attribute values within components. For example, the `hardwareAccelerated` attribute is more frequently set to `true` in 3rdPM apps. A typical scenario behind this is: to ensure compatibility with a

Table 8 Top 10 differing attribute values in four major components

Attribute Name	Total	Description	Component
resizeableActivity	593	The difference in the targetSdkVersion between the GP and 3rdPM versions of apps results in different default values being assigned to resizeableActivity.	Activity
label	485	Apps display different languages for different regions.	Activity
exported	351	The GP and 3rdPM versions of apps use different Main-Activity, resulting in differences in the exported value of MainActivity.	Activity
theme	268	Apps have different theme styles for different markets.	Activity
hardwareAccelerated	176	The 3rdPM version of apps often set hardwareAccelerated to true, while the GP version usually does not explicitly set this attribute value.	Activity
screenOrientation	165	The 3rdPM version of apps often set this attribute value to 1, indicating that the Activity is fixed in portrait mode and will not change orientation with device rotation. However, the GP version usually does not explicitly set this attribute.	Activity
configChanges	64	The GP versions of apps usually include an additional "density" value in the configChanges attribute. This indicates that the Activity will not be destroyed and recreated if the device's screen density changes. Instead, custom logic to handle the screen density change is implemented.	Activity
enableVrMode	54	The enableVrMode is a custom attribute set in the GP version of apps to support and manage virtual reality (VR) functionality provided by Google.	Activity
launchMode	19	The GP and 3rdPM versions of apps have different launch modes.	Activity
authorities	9	The GP and 3rdPM versions of apps set very similar authority attribute values. For example, only individual words have differences in letter casing.	Provider

broader range of older devices, developers on 3rdPMs might opt for a lower `targetSdkVersion` (Zimperium 2025), which disables hardware acceleration by default (Google 2025c). However, their app's monetization model is often highly dependent on video advertising SDKs that require hardware acceleration to be enabled for smooth playback (Yango-Ads 2025). This conflict between the technical default and commercial requirements compels developers to explicitly declare `android:hardwareAccelerated=true` in the manifest file to ensure their core revenue stream operates correctly.

In contrast, developers targeting Google Play typically face a different set of constraints. Due to platform policies mandating more recent `targetSdkVersion` settings (Google 2025d), hardware acceleration is often enabled by default. Furthermore, their business models may be more diversified and less dependent on video ads alone (Adjoie 2025). As a result, developers often rely on the default configuration and have no pressing need to explicitly specify the `hardwareAccelerated` attribute.

Given the potential severity of permission misconfigurations in exported components, we conducted a targeted examination of all apps in our dataset to assess the prevalence of such issues between GP and 3rdPM versions, ultimately identifying a single confirmed instance. The Dworks app⁷ declared four `Providers`, and the GP version used custom

⁷Package Name: dev.dworks.apps.anexplorer

permissions to strictly protect these `Providers`, ensuring that data could only be accessed with proper authorization. However, the 3rdPM version did not protect these `Providers`, as the permission attribute values were missing. This oversight allowed data in the 3rdPM version to be easily accessed, resulting in data leakage and posing a serious threat to user data security and privacy. While this issue appears rare in our dataset, it underscores how inconsistent configuration practices across markets—particularly those involving permission enforcement—can introduce critical security vulnerabilities.

💡 Key Insights for RQ4

Component differences between apps on GP and 3rdPM are deliberate strategies to adapt to regional technical ecosystems, business needs, and regulations. GP versions use global, GMS-centric SDKs, while 3rdPM versions employ localized alternatives (like HMS) for core functions and monetization. This extends to using `stub` components for anti-piracy and adjusting attributes (e.g., `hardwareAccelerated`) to balance compatibility with ad revenue. Critically, these inconsistencies can introduce severe security vulnerabilities, such as data leaks from inconsistent permissions, highlighting the importance of maintaining security across localized versions.

5 Discussions

5.1 Best Practices

Here we demonstrate the implications of our study on end-users, store operators, developers, and the research community.

End-users While the simplest advice is to prioritize official app stores, users often turn to 3rdPMs for various reasons. In such cases, users can adopt a more critical stance to mitigate risks: (1) **Scrutinize Permission Requests.** Be highly suspicious of apps requesting permissions that are misaligned with their core functionality. As our analysis of game apps shows (RQ3), a simple offline game asking for `ACCESS_FINE_LOCATION`, `READ_PHONE_STATE`, or `BLUETOOTH` is a strong indicator that it contains aggressive advertising or tracking SDKs. (2) **Be Wary of High-Risk Permissions.** Pay special attention to apps requesting powerful permissions like `SYSTEM_ALERT_WINDOW` (“Draw over other apps”) or Accessibility Services. Our case study on the `DroidPlugin` framework (RQ2) demonstrates that these are gateways for phishing, intrusive ads, and other malicious behavior. (3) **Leverage Privacy Tools.** Utilize built-in OS features (like the Android Privacy Dashboard) or third-party tools to review which permissions are frequently accessed and revoke those that seem unnecessary.

Store Operators (especially 3rdPMs) Instead of relying solely on traditional AV scans, market operators can implement more sophisticated, heuristic-based vetting processes based on our findings:

- **Implement Differential Analysis Heuristics:** Market operators should develop automated systems to compare submitted apps against their corresponding Google Play versions (if one exists). An app should be flagged for manual review if it exhibits a combination of high-risk indicators found in our study:
 - (a) **Mismatched Certificate Signature (High Signal):** Our findings in RQ1 show that 36.7% of app pairs have inconsistent certificates, and this is a strong predictor of repackaging.
 - (b) **Significant Permission Inflation:** The app requests a large number of permissions not found in its GP counterpart (RQ3). A high percentage of “over-claimed” (unused) permissions should further elevate the risk score.
 - (c) **Anomalous Component Profile:** The app contains suspicious unique components, such as “stub” activities/services (often linked to packers, RQ4) or dynamic loading frameworks like `DroidPlugin` (RQ2).

A rule such as “(a) AND ((b) OR (c))” could serve as a powerful heuristic to detect potentially tampered or malicious apps that warrant deeper inspection.

- **Enforce TPL Transparency:** Require developers to declare the major third-party SDKs they integrate, especially ad and analytics libraries. This would provide both operators and users with greater transparency into potential data privacy risks.

Developers Maintaining security and consistency across different distribution channels is paramount: (1) **Maintain a Unified Signing Certificate.** Always use a single, consistent developer certificate for your app across all markets. This helps establish a chain of trust and allows you, market operators, and users to more easily identify unauthorized, repackaged versions (RQ1). (2) **Audit TPLs.** Do not blindly trust integrated SDKs. Before adding a TPL, especially ad, analytics, or push-service libraries common in 3rdPMs (RQ3), conduct a security audit on the permissions it introduces and its potential vulnerabilities. If a TPL requests excessive permissions, consider finding a more privacy-respecting alternative. (3) **Ensure Consistent Security Configurations.** As shown in our analysis of the `Dworks` app (RQ4), seemingly minor configuration differences between market versions (e.g., *a missing permission on an exported component*) can introduce critical vulnerabilities. Security-critical attributes like `android:permission` and `android:exported` must be consistently and correctly enforced across all versions.

Research Community Our work offers several immediate contributions and resources for app market security studies: (1) **A Framework for Automated Differential Analysis.** The heuristics we propose for app store operators provide a concrete, evidence-based methodology that can be directly applied, customized, or extended by other researchers to perform large-scale, cross-market app analyses, thus moving beyond single-market studies. (2) **Validated Targets for Runtime Investigation.** Our static analysis identifies and validates

numerous high-risk patterns (e.g., *over-claim permissions*, *suspicious TPLs*). These identified patterns serve as vetted, high-priority targets for researchers specializing in dynamic analysis or program verification to investigate for concrete malicious behaviors at runtime. (3) Open Source Permission Mappings. As a product of permission differential analysis, it can help the research community conduct related studies, such as privacy compliance checks, security analysis, and malware detection.

5.2 Threats to Validity

Internal Validity The major threat to the “Sources Analysis” of RQ3 comes from the invocation of Non-SDK APIs within the apps (Yang et al. 2022). Calling a small subset of Non-SDK APIs in developer code or TPLs may also require permissions. Ignoring the invocation of Non-SDK APIs could lead to biases in the Sources Analysis. Fortunately, Google has implemented restrictions on access to non-SDK APIs in Android 9.0, which may help reduce threats.

External Validity Given hundreds of Android 3rdPMs, collecting apps from all 3rdPMs is impossible. Performing a differential analysis on apps with the same version code across different 3rdPMs and GP might show minor differences. To mitigate potential biases, we collected apps from as many 3rdPMs as possible and selected ten popular 3rdPMs. Therefore, our findings can represent the overall differences between the 3rdPM and GP versions of apps.

Construct Validity Our methodology depends on a range of static and dynamic analysis tools, and consequently, the accuracy of our findings is contingent upon the reliability and effectiveness of these tools. In this context, we identify two primary threats to validity:

First, the static analysis tools we used (Libd, Libradar, APKiD, MobSF, and AndroBugs) rely on signature- or rule-based detection. These signature or rule databases may be incomplete or outdated, potentially causing false negatives (e.g., *missing newly released TPLs or zero-day vulnerabilities*). In addition, MobSF and AndroBugs lack in-depth data-flow analysis, which may lead to false positives (e.g., *reporting issues that are not actually exploitable*).

Second, dynamic analysis is also limited in accuracy and coverage. For example, we infer the deployment of signature verification based on whether an app crashes after repackaging, but such crashes may be caused by unrelated system errors, potentially leading to an overestimation of its prevalence. In addition, FRIDA-DEXDump fails outright when encountering complex packer schemes, which is not a typical false negative but rather a fundamental limitation of the tool’s capabilities.

Conclusion Validity Our conclusions face two primary threats. First, our findings depend on the reliability of automated tools. For instance, our conclusions about the prevalence of certain vulnerabilities (RQ2) rely on scanners like MobSF and AndroBugs. Since these tools can produce false positives, the measured frequency of a vulnerability (e.g., *insecure data storage*) might be inflated. Thus, any conclusion about its prevalence is contingent on the scanner’s accuracy. Second, we are cautious about generalizing from small samples. While

a specific case like the insecure `Provider` in the `Dworks` app (RQ4) highlights a critical risk, its uniqueness in our dataset prevents us from concluding it is a widespread problem.

5.3 Future Work

This study primarily focuses on the differences at the code level between the GP and 3rdPM versions of apps, such as protection mechanisms, vulnerabilities, and permissions. However, our findings, particularly the reduced deployment of packers and signature verification in the GP version of apps, reveal a critical and underexplored area of risk: the security of valuable in-app assets. Many modern apps bundle valuable, proprietary resources directly within their APKs. These can include machine learning models (e.g., *for on-device image recognition*), sensitive configuration files containing API keys, proprietary business logic, or even pre-packaged databases. The weaker protection measures we observed in many Google Play apps create a significant risk that adversaries could more easily reverse-engineer these apps to extract, steal, or tamper with these assets. Such actions could lead to severe security issues, including intellectual property theft (e.g., *extracting a competitor's AI model*), sensitive data leakage (e.g., *stealing API keys*), and malicious tampering (e.g., *modifying model parameters or business logic to alter app behavior*).

Therefore, a systematic investigation into the differential risk to these in-app assets represents a worthy direction for future research. To effectively identify and assess these risks, future work could employ a multi-faceted detection strategy:

- **Static Analysis for Asset Identification:** Automatically scan APK files to identify and categorize valuable assets. This would involve searching for common file extensions (e.g., `.tflite`, `.onnx` for AI models; `.json`, `.xml` for configuration files) and using pattern matching or entropy analysis to detect encrypted or obfuscated sensitive files.
- **Dynamic Analysis for Asset Usage Monitoring:** Instrument and run apps in a controlled environment to monitor how and when these assets are loaded and used. This can help confirm their value and identify weak points in the runtime protection logic, such as moments when assets are decrypted in memory.
- **Model-Specific Security Audits:** For apps containing AI models, specific detection techniques could be applied. This includes analyzing the model structure for vulnerabilities, checking for the presence of model watermarking or encryption, and performing model extraction attacks to measure the ease of theft.

By combining these strategies, subsequent research can systematically identify apps with exposed assets, quantify the increased risk due to inconsistent protection levels across markets, and contribute to a more comprehensive understanding of the cross-market security ecosystem.

6 Related Work

In this section, we review the related work on app market measurement. We categorize prior studies into three main themes: cross-market app analysis, app behavior and quality analysis, and security and privacy analysis.

Cross-Market App Analysis A few studies have focused on comparative studies across different markets. The most relevant work was conducted by Wang et al. (2018). They compared 16 Chinese app markets and Google Play, revealing differences in developer behavior across these app markets, including aspects like code maintenance and the use of third-party services. Furthermore, Steinböck et al. (2024) focused on cross-platform analysis, successfully identifying 3,322 cross-platform app pairs from 10,000 popular apps each on the Apple App Store and Google Play. Their work revealed differences between the two platforms in developer policies, data collection, and permission management, further indicating that the Android ecosystem still has vulnerabilities in privacy protection and security regulation.

App Behavior and Quality Analysis Many works have focused on measuring the behavior of apps and developers within specific app markets. For instance, Viennot et al. (2014) used PlayDrone to download over 1.1 million Android apps from Google Play and investigated four valuable questions to help improve app content quality. Several studies have leveraged user reviews to analyze app quality. Chen et al. (2014b) proposed AR-Miner to mine and analyze user reviews to identify valuable feedback for developers. Similarly, Hu et al. (2021) developed CHAMP, a tool using text mining and NLP to detect policy-violating behaviors from user reviews. Other works have focused on specific app characteristics. Cai et al. (2019) conducted a large-scale, longitudinal study on compatibility issues in the Android ecosystem. Vásquez et al. (2013) investigated how the fault- and change-proneness of APIs affect the success of apps. Hecht et al. (2015) employed PAPRIKA to analyze the evolution of mobile app quality, identifying several Android-specific antipatterns. Zhou et al. (2020) studied diehard apps and found that about 21% of analyzed apps employed various diehard methods. Liu et al. (2019) introduced DAPANDA to automatically detect aggressive push notifications in Android apps.

Security and Privacy Analysis Another line of research focuses on security and privacy issues in mobile apps. Some works have investigated the deployment of app protection measures. For example, Berlato and Ceccato (2020) analyzed 38,323 apps on Google Play and found that 59% of them neither implemented anti-debugging nor anti-tampering protections. Dong et al. (2022) proposed PackDiff to expose issues with commercial app packers through dynamic analysis. Wang et al. (2019) conducted a large-scale systematic measurement study on app signing issues. Others have focused on detecting malicious behaviors. Chen et al. (2014a) and Wang et al. (2015) proposed different two-phase methods (based on method similarity and static semantic features, respectively) to detect app clones in Android markets. Li et al. (2015) developed IccTA to detect data leaks between components in Android apps, uncovering significant privacy violations. Dong et al. (2018) created FraudDroid to detect mobile ad fraud. Tang et al. (2019) analyzed popular APM tools and revealed how they are used to collect user privacy information.

The prior work provides valuable insights into the Android app ecosystem, covering cross-market developer behavior, app quality, and various security and privacy threats. However, most studies either focus on a single market (predominantly Google Play) or conduct high-level comparisons across markets without delving into the specific implementation differences of the same app. To the best of our knowledge, a comprehensive and multi-perspective differential analysis that systematically compares the security threats,

protection mechanisms, permission configurations, and component definitions between GP and 3rdPM versions of the same app remains largely unexplored. This work aims to fill that gap by providing the first in-depth, multi-dimensional investigation of cross-market inconsistencies, shedding light on the underlying risks and revealing new insights into the fragmented nature of Android app distribution.

7 Conclusion

Even if Google Play and third-party market versions of apps have the same version code, they might have differences in implementations. In this work, we systematically analyzed the differences between apps with the same version code from four points of view, including app protection, security threats, permission usage, and major components. A series of captivating and valuable insights have been revealed, which can help developers and app markets provide improvement directions and guidance, enhancing the quality of the Android apps ecosystem and user experience.

Author Contributions Shishuai Yang analyzed the experimental data and wrote the manuscript. Ruoyang Lin conducted the experiments. Jialong Guo assisted in data analysis. Guangdong Bai contributed to manuscript revision. Yujia Luo assisted in setting up the experimental environment. Wenrui Diao proposed the idea and led the project.

Funding This work was supported by Taishan Young Scholar Program of Shandong Province (Grant No. tsqn202211001) and Natural Science Foundation of Shandong Province (Grant No. ZR2023MF043 and ZR2025MS991).

Data Availability The raw data obtained and analyzed during the current study are available at <https://doi.org/10.5281/zenodo.13232037>.

Declarations

Ethical Approval Not applicable.

Informed Consent Not applicable.

Conflict of Interest The authors declared that they have no conflict of interest.

Clinical Trial Number Not applicable.

References

- Adjoe (2025) Mobile App Monetization Strategies: Maximize Revenue in 2025. <https://adjoe.io/blog/app-monetization-strategies/>
- Ajin-Abraham (2023) MobSF. <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- Ajin-Abraham (2024) Automated Mobile Application Security Assessment with MobSF. <https://www.slideshare.net/ajin25/automated-mobile-application-security-assessment-with-mobsf>
- Androguard (2023) Androguard. <https://github.com/androguard/androguard>
- AVClass (2025) AVClass. <https://github.com/malicialab/avclass>
- Berlato S, Ceccato M (2020) A large-scale study on the adoption of anti-debugging and anti-tampering protections in Android apps. *J Inf Secur Appl*. <https://doi.org/10.1016/j.jisa.2020.102463>
- ByteDance (2025) Pangle. <https://www.pangleglobal.com/>

- Cai H, Zhang Z, Li L, Fu X (2019) A Large-Scale Study of Application Incompatibilities in Android. In: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), Beijing, China, July 15-19, 2019
- Check-Point (2025) SimBad: A Rogue Adware Campaign On Google Play. <https://research.checkpoint.com/2019/simbad-a-rogue-adware-campaign-on-google-play/>
- Chen K, Liu P, Zhang Y (2014a) Achieving Accuracy and Scalability Simultaneously in Detecting Application Clones on Android Markets. In: Proceedings of the 36th International Conference on Software Engineering (ICSE), Hyderabad, India, May 31 - June 07, 2014
- Chen N, Lin J, Hoi SCH, Xiao X, Zhang B (2014b) AR-Miner: Mining Informative Reviews for Developer from Mobile App MarketPlace. In: Proceedings of the 36th International Conference on Software Engineering (ICSE), Hyderabad, India, May 31 - June 07, 2014
- Dai D, Li R, Tang J, Davanian A, Yin H (2020) Parallel Space Traveling: A Security Analysis of App-Level Virtualization in Android. In: Proceedings of the 25th ACM Symposium on Access Control Models and Technologies (SACMAT), 2020, Barcelona, Spain, June 10-12, 2020
- Dietz M, Shekhar S, Pisetsky Y, Shu A, Wallach DS (2011) QUIRE: Lightweight Provenance for Smart Phone Operating Systems. In: Proceedings of the 20th USENIX Security Symposium (USENIX-SEC), San Francisco, CA, USA, August 8-12, 2011
- Dong F, Wang H, Li L, Guo Y, Bissyandé TF, Liu T, Xu G, Klein J (2018) FraudDroid: Automated Ad Fraud Detection for Android Apps. In: Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), Lake Buena Vista, FL, USA, November 04-09, 2018
- Dong Z, Liu H, Wang L, Luo X, Guo Y, Xu G, Xiao X, Wang H (2022) What Did You Pack in My App? A Systematic Analysis of Commercial Android Packers. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), Singapore, Singapore, November 14-18, 2022
- DroidPlugin (2025) DroidPlugin. <https://github.com/DroidPluginTeam/DroidPlugin>
- Erik-Derr (2023) Aexplorer. <https://github.com/reddr/explorer>
- Facebook (2023) Meta Audience Network. <https://developers.facebook.com/docs/audience-network/>
- Gao J, Li L, Kong P, Bissyandé TF, Klein J (2021) Understanding the evolution of Android app vulnerabilities. IEEE Trans Reliab. <https://doi.org/10.1109/TR.2019.2956690>
- Google (2023a) Android Cloud to Device Messaging (C2DM). <https://www.vogella.com/tutorials/GoogleCloudMessaging/article.html>
- Google (2023b) Declare Permissions for Your App. <https://support.google.com/googleplay/android-developer/answer/9214102>
- Google (2023c) Firebase. <https://firebase.google.com/>
- Google (2023d) Google GMS Ads. <https://developers.google.com/android/reference/com/google/android/gms/ads/package-summary>
- Google (2023e) Google GMS Measurement. <https://developers.google.com/android/reference/com/google/android/gms/measurement/package-summary>
- Google (2023f) Intellectual Property. <https://support.google.com/googleplay/android-developer/answer/9888072>
- Google (2023g) Permissions on Android. <https://developer.android.com/guide/topics/permissions/overview>
- Google (2023h) Sign Your App. <https://developer.android.com/studio/publish/app-signing>
- Google (2024) Developer Policy Center. <https://play.google/developer-content-policy/>
- Google (2025a) Best Practices for Unique Identifiers. <https://github.com/DroidPluginTeam/DroidPlugin>
- Google (2025b) FileProvider. <https://developer.android.com/reference/androidx/core/content/FileProvider>
- Google (2025c) Hardware Acceleration. <https://developer.android.com/topic/performance/hardware-accel>
- Google (2025d) Target API Level Requirements for Google Play Apps. <https://support.google.com/googleplay/android-developer/answer/11926878>
- Hecht G, Benomar O, Rouvoy R, Moha N, Duchien L (2015) Tracking the Software Quality of Android Applications Along Their Evolution. In: Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE), Lincoln, NE, USA, November 9-13, 2015
- Hluwa (2023) FRIDA-DEXDump. <https://github.com/hluwa/frida-dexdump>
- Hu Y, Wang H, Ji T, Xiao X, Luo X, Gao P, Guo Y (2021) CHAMP: Characterizing Undesired App Behaviors from User Comments based on Market Policies. In: Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE), Madrid, Spain, 22-30 May 2021
- HUAWEI (2023) HMS Core. <https://developer.huawei.com/consumer/en/hms/>
- IBM (2023) Creating Application Stubs in Tests. <https://www.ibm.com/docs/en/rtw/9.5?topic=tests-creating-application-stubs-in>
- JoMingyu (2023) Google-Play-Scraper. <https://github.com/JoMingyu/google-play-scraper>

- Li L, Bartel A, Bissyandé TF, Klein J, Traon YL, Arzt S, Rasthofer S, Bodden E, Octeau D, McDaniel PD (2015) IccTA: Detecting Inter-Component Privacy Leaks in Android Apps. In: Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE), Florence, Italy, May 16-24, 2015
- Li M, Wang W, Wang P, Wang S, Wu D, Liu J, Xue R, Huo W (2017) LibD: Scalable and Precise Third-Party Library Detection in Android Markets. In: Proceedings of the 39th International Conference on Software Engineering (ICSE), Buenos Aires, Argentina, May 20-28, 2017
- Liran-Tal (2025) <https://snyk.io/articles/secure-sdlc/>. <https://snyk.io/articles/secure-sdlc/>
- Liu T, Wang H, Li L, Bai G, Guo Y, Xu G (2019) DaPanda: Detecting Aggressive Push Notifications in Android Apps. In: Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), San Diego, CA, USA, November 11-15, 2019
- Lookout (2023) Pinduoduo App Report. <https://www.lookout.com/documents/threat-reports/us/lookout-tg-pinduoduo.pdf>
- Ma Z, Wang H, Guo Y, Chen X (2016) LibRadar: Fast and Accurate Detection of Third-party Libraries in Android Apps. In: Proceedings of the 38th International Conference on Software Engineering (ICSE), Austin, TX, USA, May 14-22, 2016
- MobSF (2025) Mobile Security Framework (MobSF) Documentation. <https://mobsf.github.io/docs/>
- OWASP (2023) OWASP Mobile Top 10. <https://owasp.org/www-project-mobile-top-10/>
- OWASP (2024) Testing Tools. <https://mas.owasp.org/MASTG/tools/>
- OWASP (2025) OWASP MASVS. <https://mas.owasp.org/MASVS/>
- Pinduoduo (2023) Pinduoduo App Website. <https://en.pinduoduo.com/>
- Raccoon (2023) Raccoon. <https://raccoon.onyxbits.de/>
- Raccoon (2024) Privacy Statement. <https://raccoon.onyxbits.de/privacy/>
- Red-Naga (2023) APKiD. <https://github.com/rednaga/APKiD>
- Steinböck M, Bleier J, Rainer M, Urban T, Utz C, Lindorfer M (2024) Comparing Apples to Androids: Discovery, Retrieval, and Matching of iOS and Android Apps for Cross-Platform Analyses. In: Proceedings of the 21st IEEE/ACM International Conference on Mining Software Repositories (MSR), Lisbon, Portugal, April 15-16, 2024
- Tang Y, Zhan X, Zhou H, Luo X, Xu Z, Zhou Y, Yan Q (2019) Demystifying Application Performance Management Libraries for Android. In: Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), San Diego, CA, USA, November 11-15, 2019
- Umeng (2025) Umeng. <https://www.umeng.com/>
- Unity (2023) Unity Ads. <https://docs.unity3d.com/550/Documentation/Manual/UnityAdsHowTo.html>
- URORA (2023) JPush. <https://www.jiguang.cn/en/push>
- Vásquez ML, Bavota G, Bernal-Cárdenas C, Penta MD, Oliveto R, Poshyanyk D (2013) API Change and Fault Proneness: A Threat to the Success of Android Apps. In: Proceedings of the 9th ACM joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE), Saint Petersburg, Russian Federation, August 18-26, 2013
- Victor-M-Alvarez (2023) Yara Rules. <https://yara.readthedocs.io/en/stable/index.html>
- Viennot N, Garcia E, Nieh J (2014) A Measurement Study of Google Play. In: Proceedings of the 34th International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS), Austin, TX, USA, June 16-20, 2014
- VirusTotal (2023) VirusTotal. <https://www.virustotal.com/gui/home/upload>
- Wang H, Guo Y, Ma Z, Chen X (2015) WuKong: A Scalable and Accurate Two-Phase Approach to Android App Clone Detection. In: Proceedings of the 24th International Symposium on Software Testing and Analysis (ISSTA), Baltimore, MD, USA, July 12-17, 2015
- Wang H, Liu H, Xiao X, Meng G, Guo Y (2019) Characterizing Android App Signing Issues. In: Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), San Diego, CA, USA, November 11-15, 2019
- Wang H, Liu Z, Liang J, Vallina-Rodriguez N, Guo Y, Li L, Tapiador J, Cao J, Xu G (2018) Beyond Google Play: A Large-Scale Comparative Study of Chinese Android App Markets. In: Proceedings of the 18th Internet Measurement Conference (IMC), Boston, MA, USA, October 31 - November 02, 2018
- Yang S, Li R, Chen J, Diao W, Guo S (2022) Demystifying Android Non-SDK APIs: Measurement and Understanding. In: Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA, May 25-27, 2022
- Yang S, Bai G, Lin R, Guo J, Diao W (2024) Beyond the Horizon: Exploring Cross-Market Security Discrepancies in Parallel Android Apps. In: Proceedings of the 35th IEEE International Symposium on Software Reliability Engineering (ISSRE), Tsukuba, Japan, October 28-31, 2024
- Yango-Ads (2025) Monetizing Your Android App: Strategies That Work in 2025. <https://yango-ads.com/blog/monetizing-your-android-app-strategies-work-2025>

- Yu-Cheng (2023) AndroBugs. https://github.com/AndroBugs/AndroBugs_Framework
- Yu-Cheng (2024) Responsible Disclosure of AndroBugs. <https://www.androbugs.com/#hof>
- Zhan X, Fan L, Liu T, Chen S, Li L, Wang H, Xu Y, Luo X, Liu Y (2020) Automated Third-Party Library Detection for Android Applications: Are We There Yet? In: Proceedings of 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), Melbourne, Australia, September 21–25, 2020
- Zhang Y, Luo X, Yin H (2015) DexHunter: Toward Extracting Hidden Code from Packed Android Applications. In: Proceedings of 20th European Symposium on Research in Computer Security (ESORICS), Vienna, Austria, September 21–25, 2015
- Zhou H, Wang H, Zhou Y, Luo X, Tang Y, Xue L, Wang T (2020) Demystifying Diehard Android Apps. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), Melbourne, Australia, September 21–25, 2020
- Zimperium (2025) Global Mobile Threat Report. <https://lp.zimperium.com/hubfs/Reports/2025%20Global%20Mobile%20Threat%20Report.pdf>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Shishuai Yang is a Lecturer in the School of Computer Science, Zhengzhou University of Aeronautics, Zhengzhou, China. He received the Ph.D. degree from Shandong University and his B.Sc. degree from Henan University, China. His research interests include mobile security and IoT security.



Ruoyan Lin received the B.Sc. degree in Cyberspace Security from Shandong University, Qingdao, China, in 2023. She is currently working toward the Master's degree at the School of Cyber Science and Technology, Shandong University, Qingdao, China. Her research interests include mobile security.



Jialong Guo received the B.E. degree in Information Security from Zhengzhou University. He is currently working toward the Ph.D. degree at the School of Cyber Science and Technology, Shandong University, Qingdao, China. His research interests include IoT firmware security and fuzzing.



Guangdong Bai is an Associate Professor in the School of Electrical Engineering and Computer Science at the University of Queensland (UQ), Brisbane, Australia. He received his Ph.D. degree from the National University of Singapore and his M.S. and B.S. degrees from Peking University, China. He leads the TrustLab at UQ Cyber Security and Software Engineering and is an Associate Editor of IEEE Transactions on Dependable and Secure Computing. His research interests include Security and Privacy, Software Engineering, and Formal Methods.



Yujia Luo is a Laboratory Engineer in the School of Cyber Science and Technology, Shandong University, Qingdao, China. She received the Master degree in Communication and Information Systems from the University of Electronic Science and Technology of China. Her main research interests are computer networks and data communication security.



Wenrui Diao is a Professor in the School of Cyber Science and Technology, Shandong University, Qingdao, China. He received the Ph.D. degree in Information Engineering from The Chinese University of Hong Kong in 2017. His research interests include mobile security and IoT security. He received the 2019 ACM SIGSAC China Rising Star Award, ACM CCS 2024 Distinguished Paper Award, and Best Practical Paper in Cybersecurity Award 2025.

Authors and Affiliations

Shishuai Yang¹ · Ruoyan Lin² · Jialong Guo² · Guangdong Bai³ · Yujia Luo² · Wenrui Diao² 

✉ Wenrui Diao
diaowenrui@link.cuhk.edu.hk

Shishuai Yang
shishuai@zua.edu.cn

Ruoyan Lin
ruoyan@mail.sdu.edu.cn

Jialong Guo
guoxb@mail.sdu.edu.cn

Guangdong Bai
g.bai@uq.edu.au

Yujia Luo
luoyuj@sdu.edu.cn

¹ School of Computer Science, Zhengzhou University of Aeronautics, Zhengzhou, China

² School of Cyber Science and Technology, Shandong University, Qingdao, China

³ School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, Australia